

A Geometric Nearest Point Algorithm for the Efficient Solution of the SVM Classification Task

IEEE TRANSACTIONS ON NEURAL NETWORKS
VOL. 18, NO. 5, SEPTEMBER 2007

Michael E. Mavroforakis, Margaritis Sdalis, and Sergios
Theodoridis

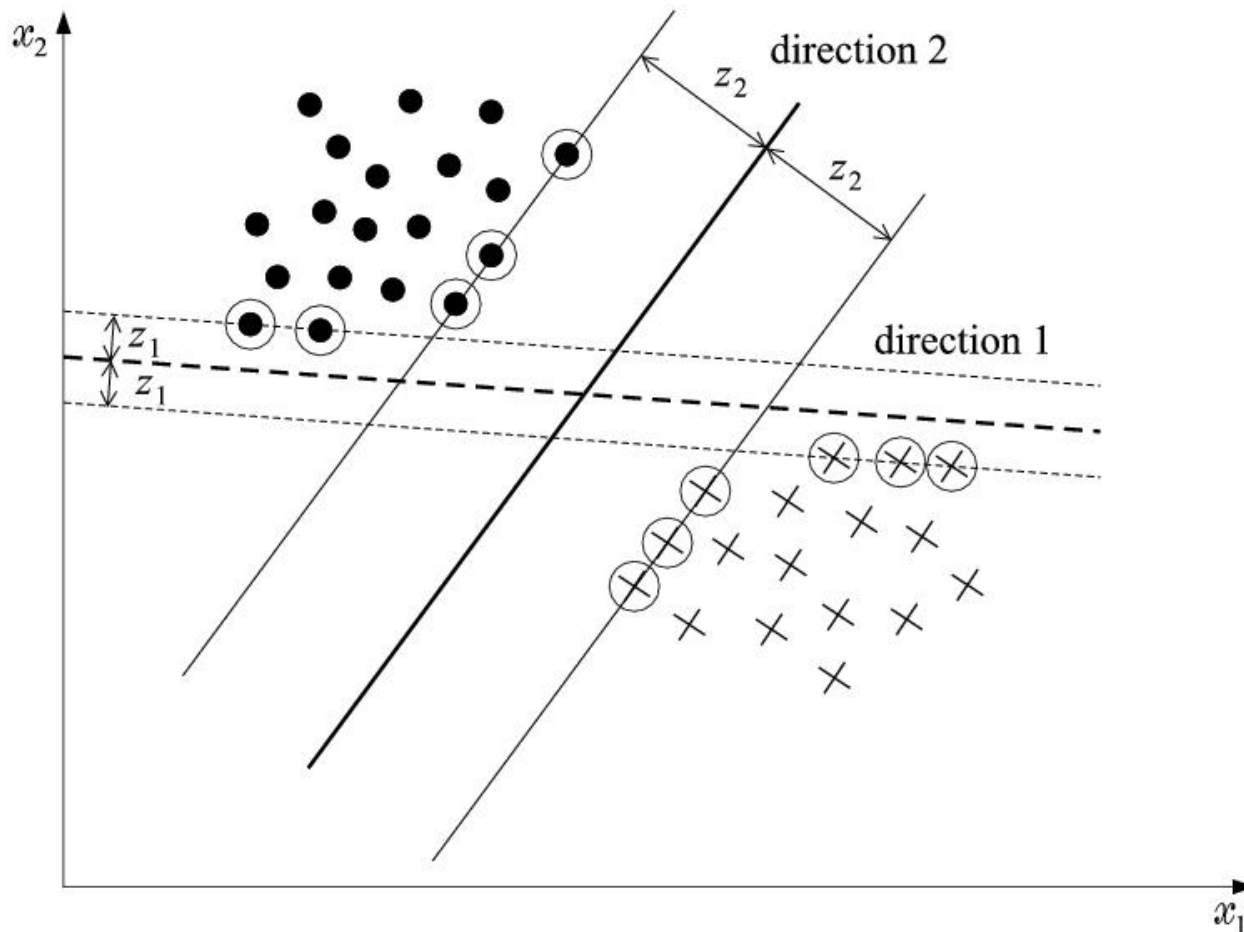
Presenter: Chung, Wei-Chun
Date: May 7, 2008

Abstract

- Geometric methods are very intuitive and provide a theoretically solid approach to many optimization problems. One such optimization task is the support vector machine (SVM) classification, which has been the focus of intense theoretical as well as application-oriented research in machine learning. In this letter, the incorporation of recent results in reduced convex hulls (RCHs) to a nearest point algorithm (NPA) leads to an elegant and efficient solution to the SVM classification task, with encouraging practical results to real-world classification problems, i.e., linear or nonlinear and separable or nonseparable.

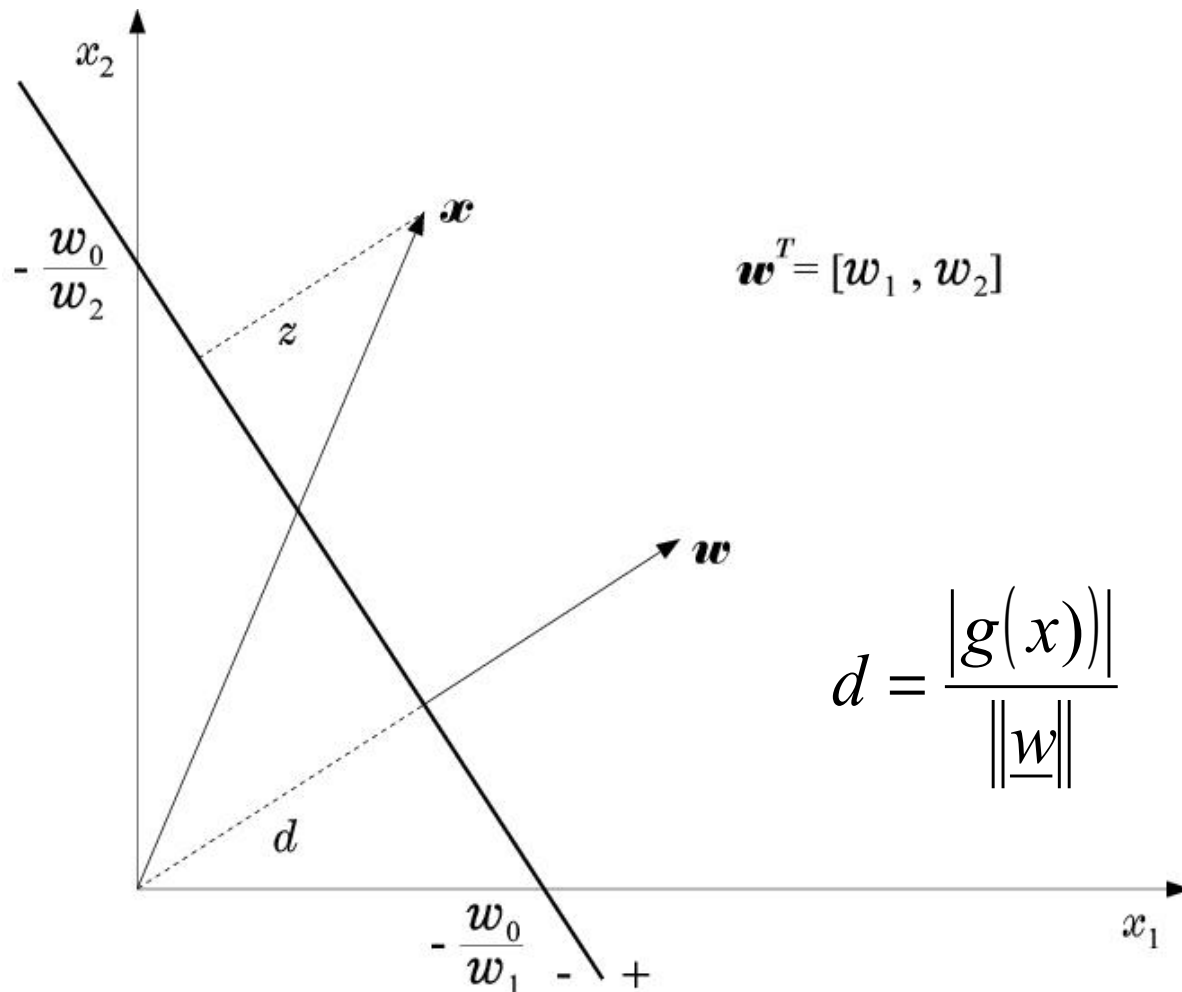
SVM (1/6)

- The maximum margin interpretation.
- It is a typical constrained optimization problem



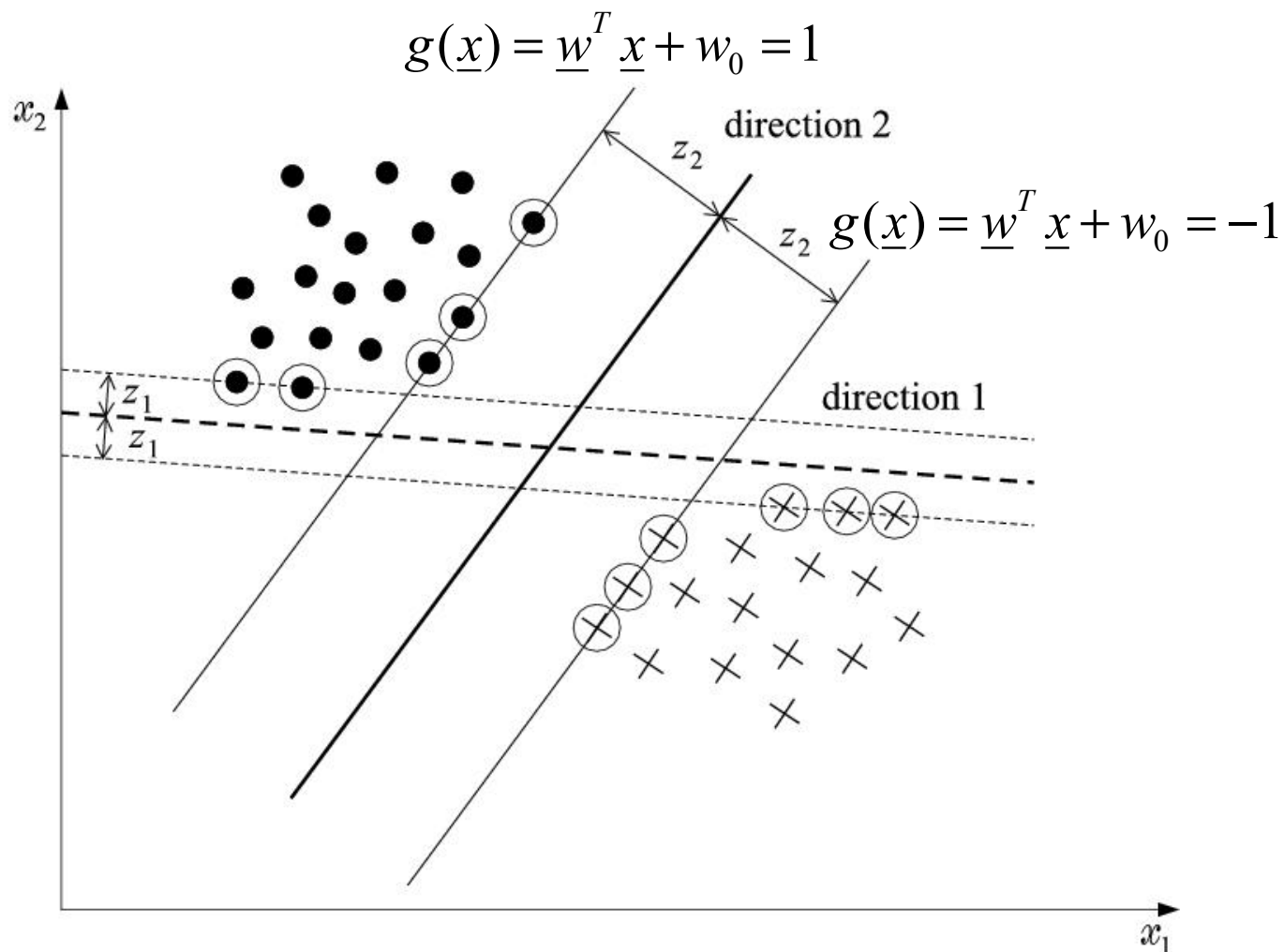
SVM (2/6)

- For hyperplane: $g(\underline{x}) = \underline{w}^T \underline{x} + w_0 = 0$



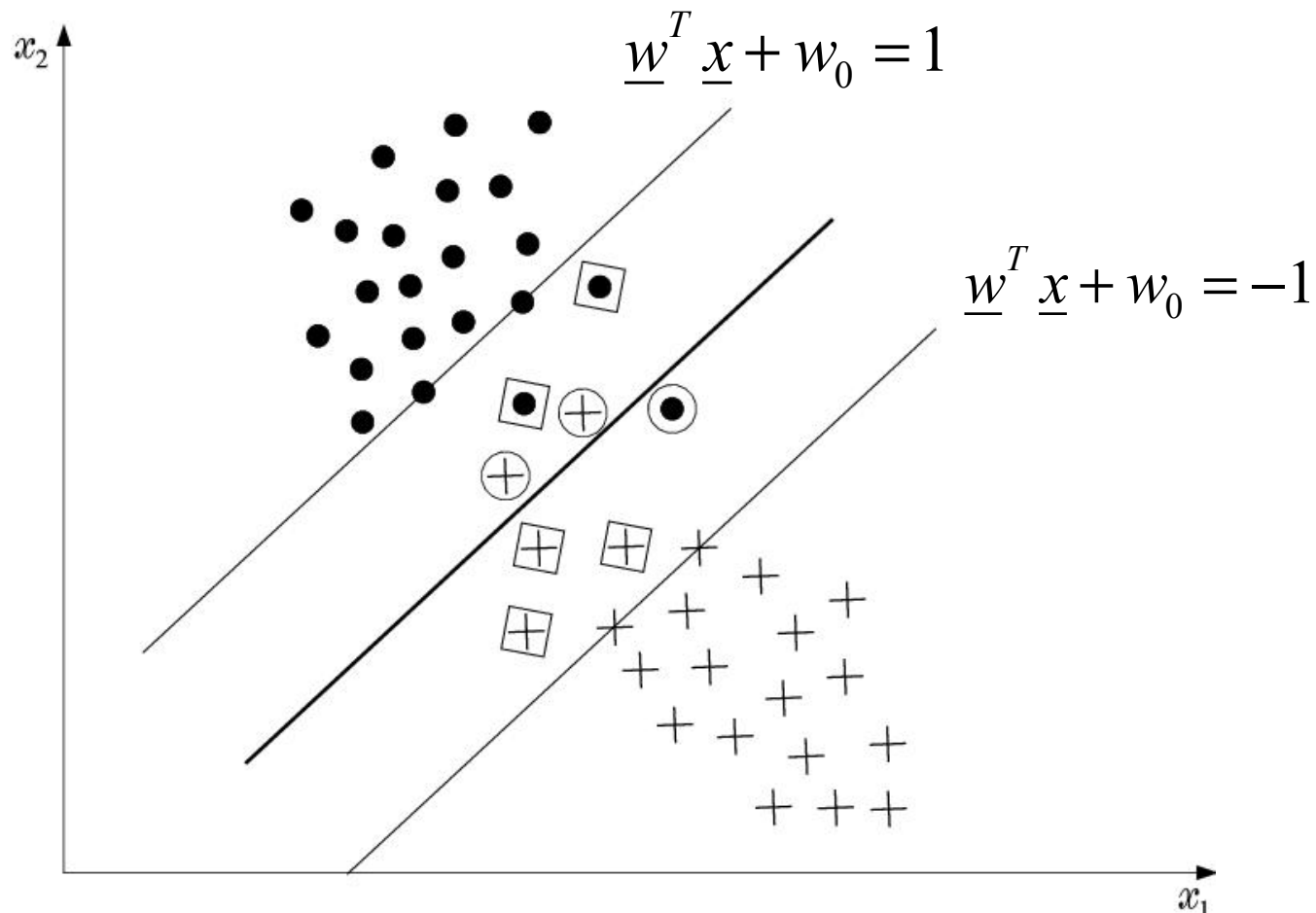
SVM (3/6)

- The separable class case:



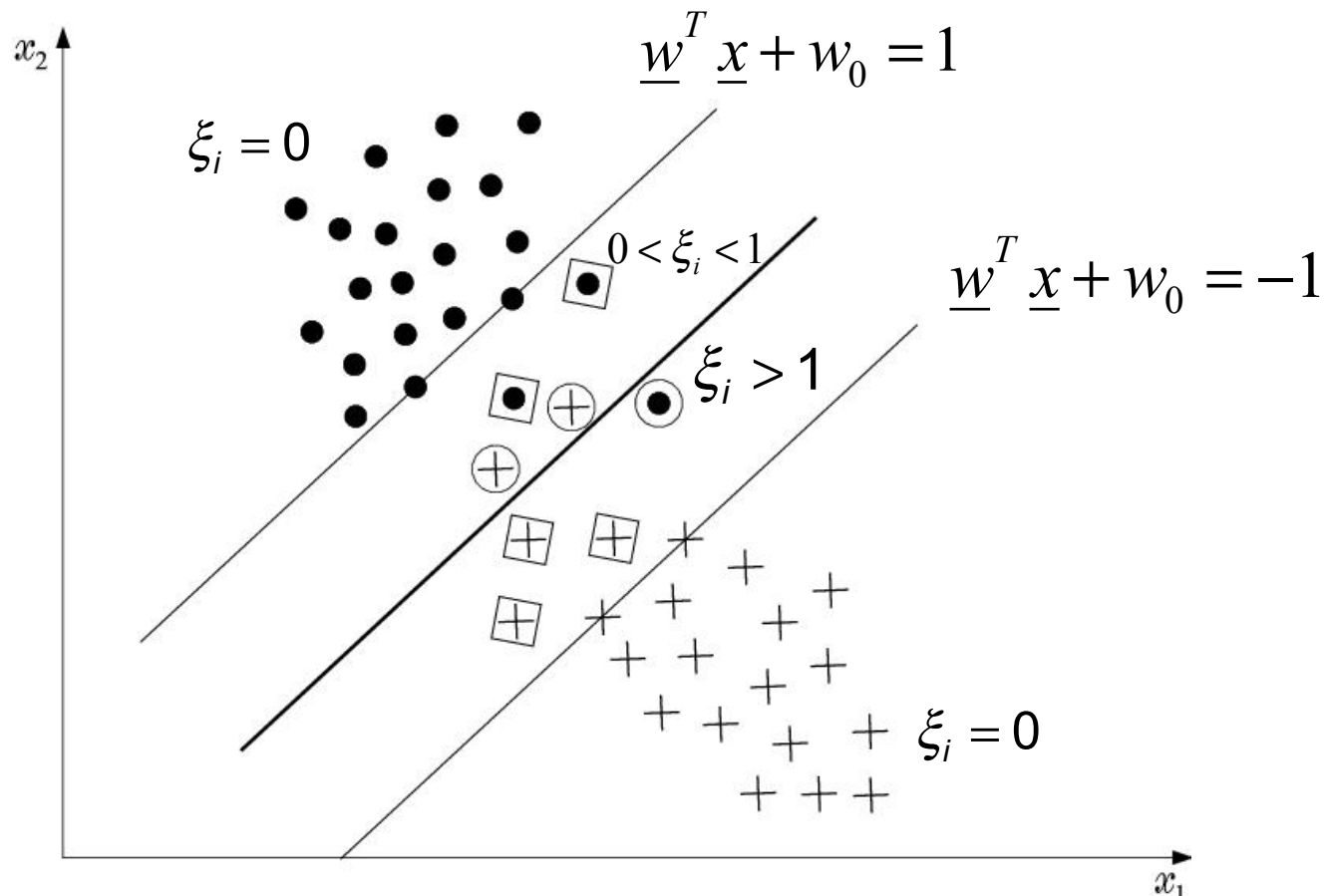
SVM (4/6)

- The non-separable class case:



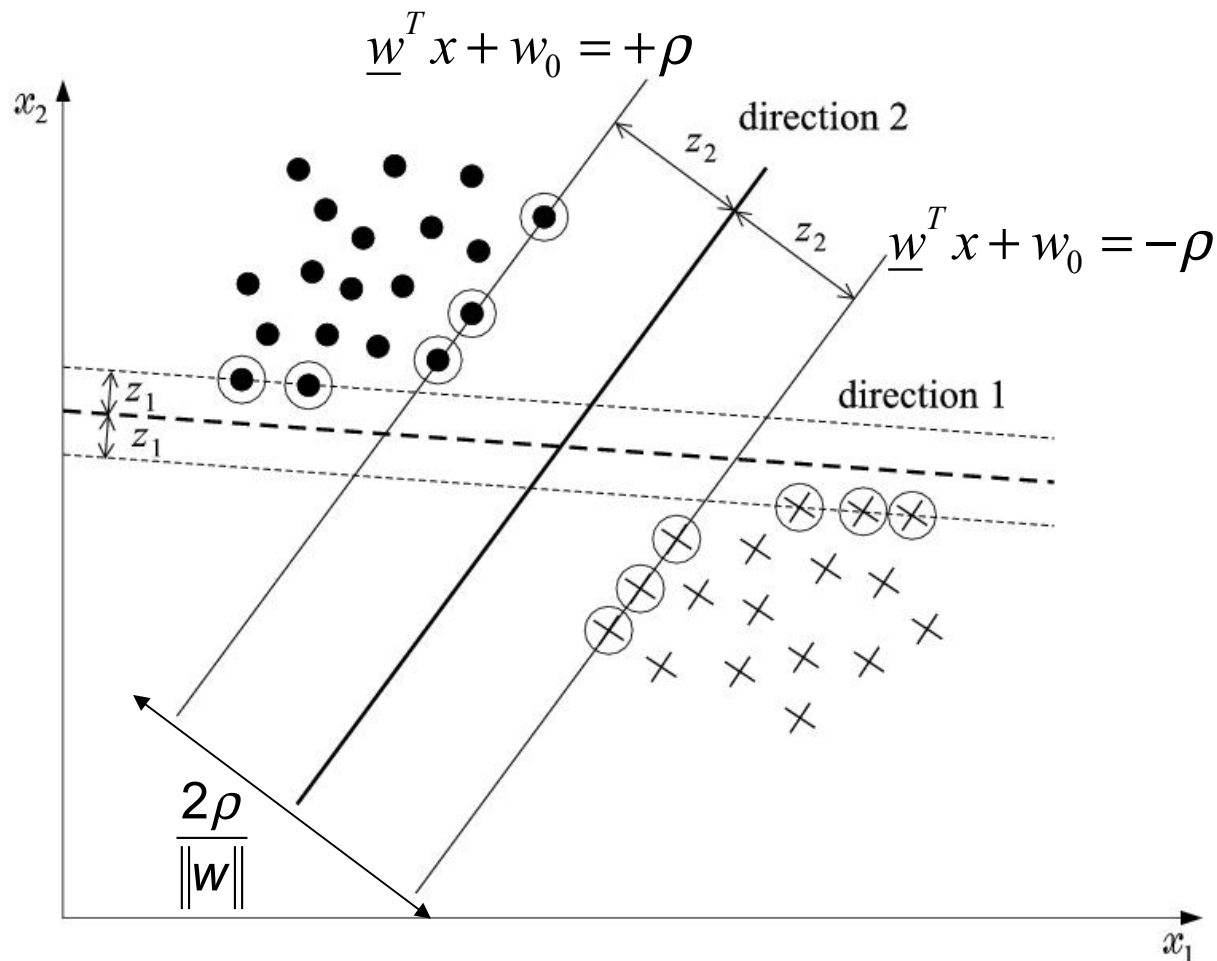
SVM (5/6)

- With slack variable: $y_i [\underline{w}^T \underline{x}_i + w_0] \geq 1 - \xi_i$
 - try to minimize margin errors



SVM (6/6)

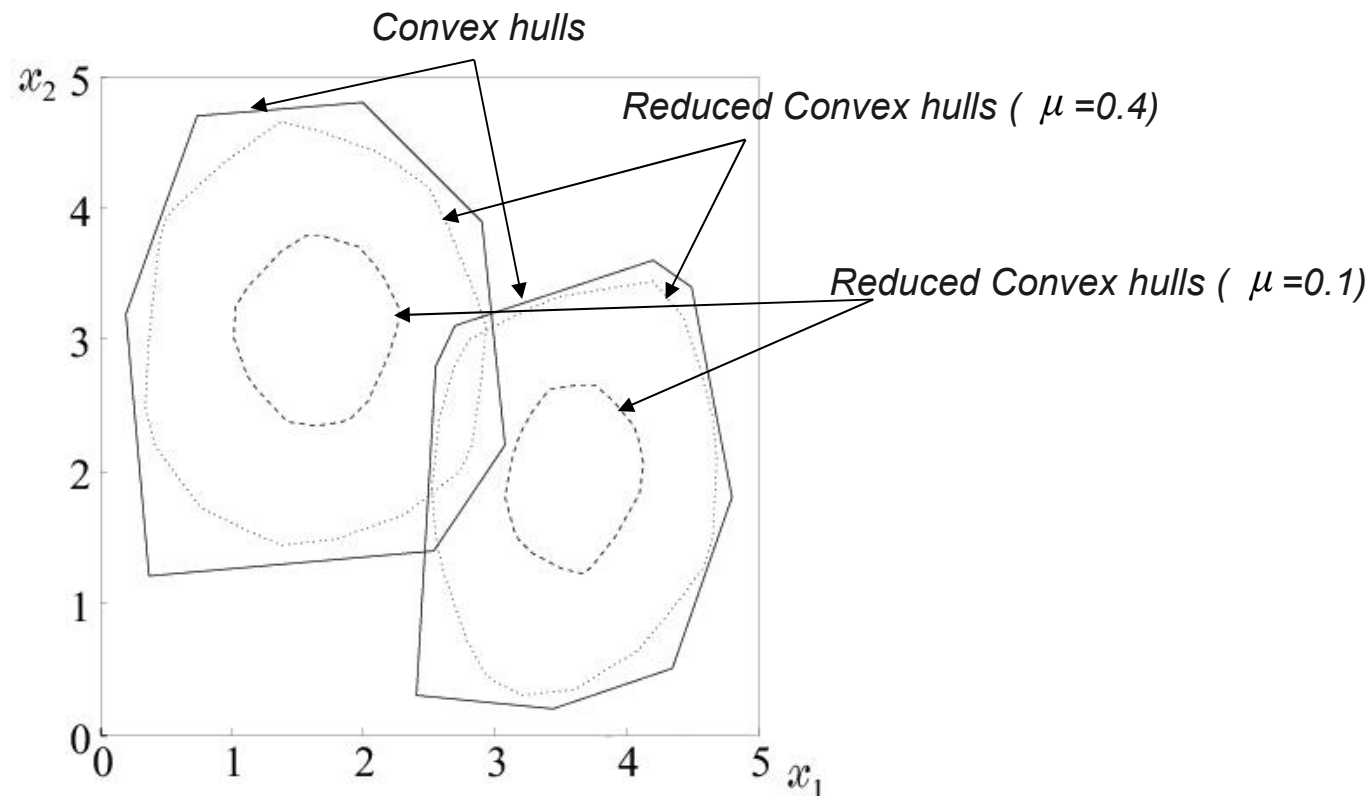
- ν -SVM: minimize $J(\underline{w}) = \|\underline{w}\|^2 - 2\rho + \mu \sum_{i=1}^N \xi_i$



RCH (1/2)

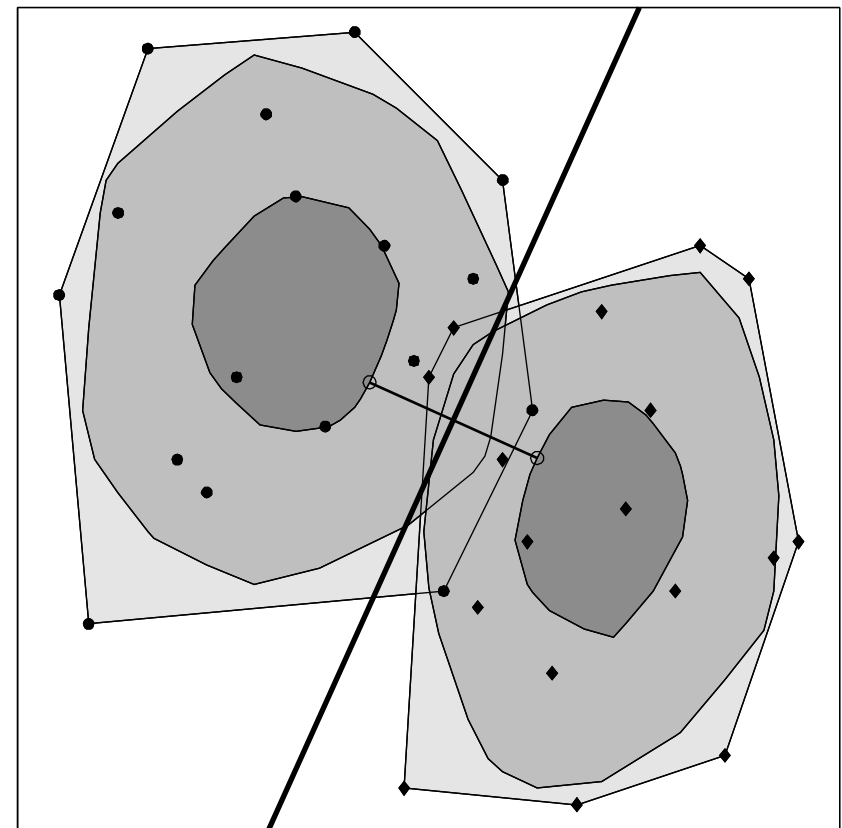
- Reduced Convex Hull

$$- R(X, \mu) = \left\{ w : w = \sum_{i=1}^k a_i x_i, x_i \in X, \sum_{i=1}^k a_i = 1, 0 \leq a_i \leq \mu \right\}$$



RCH (2/2)

- Subject to: $(x_1^*, x_2^*) = \arg \min(\|x_1 - x_2\|)$
 - where $x_1 \in R(X_1, \mu)$ and $x_2 \in R(X_2, \mu)$.
- With:
 - $R(X_1, \mu) \cap R(X_2, \mu) = \emptyset$.
 - This is clearly a nearest point problem (NPP) and it is equivalent to the minimum norm problem (MNP)



Gilbert's Algorithm

- For the Minimum Norm Problem (MNP) for Convex Hulls

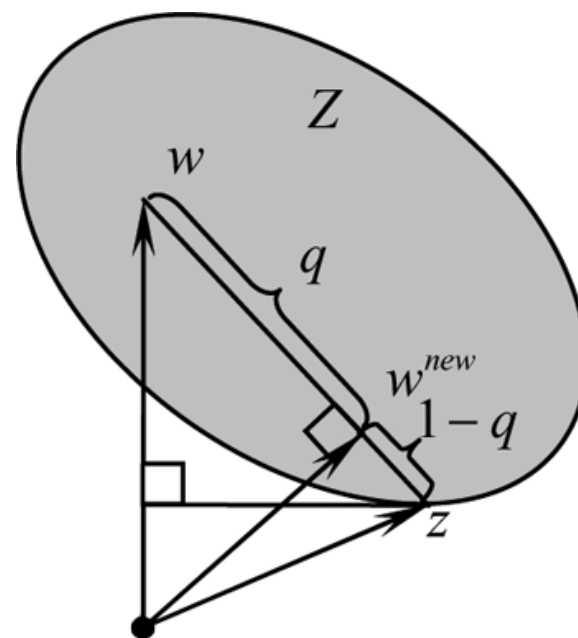
-

- Algorithm:

Step 1: Choose $w \in Z$.

Step 2: Find the point $z \in Z$ with the minimum projection onto the direction of w . If $\|w\| \cong \|z\|$, set $w^* \leftarrow z$; stop.

Step 3: Find the point w^{new} of the line segment $[w, z]$, with minimum norm (closest to the origin). Set $w \leftarrow w^{\text{new}}$ and go to Step 2.



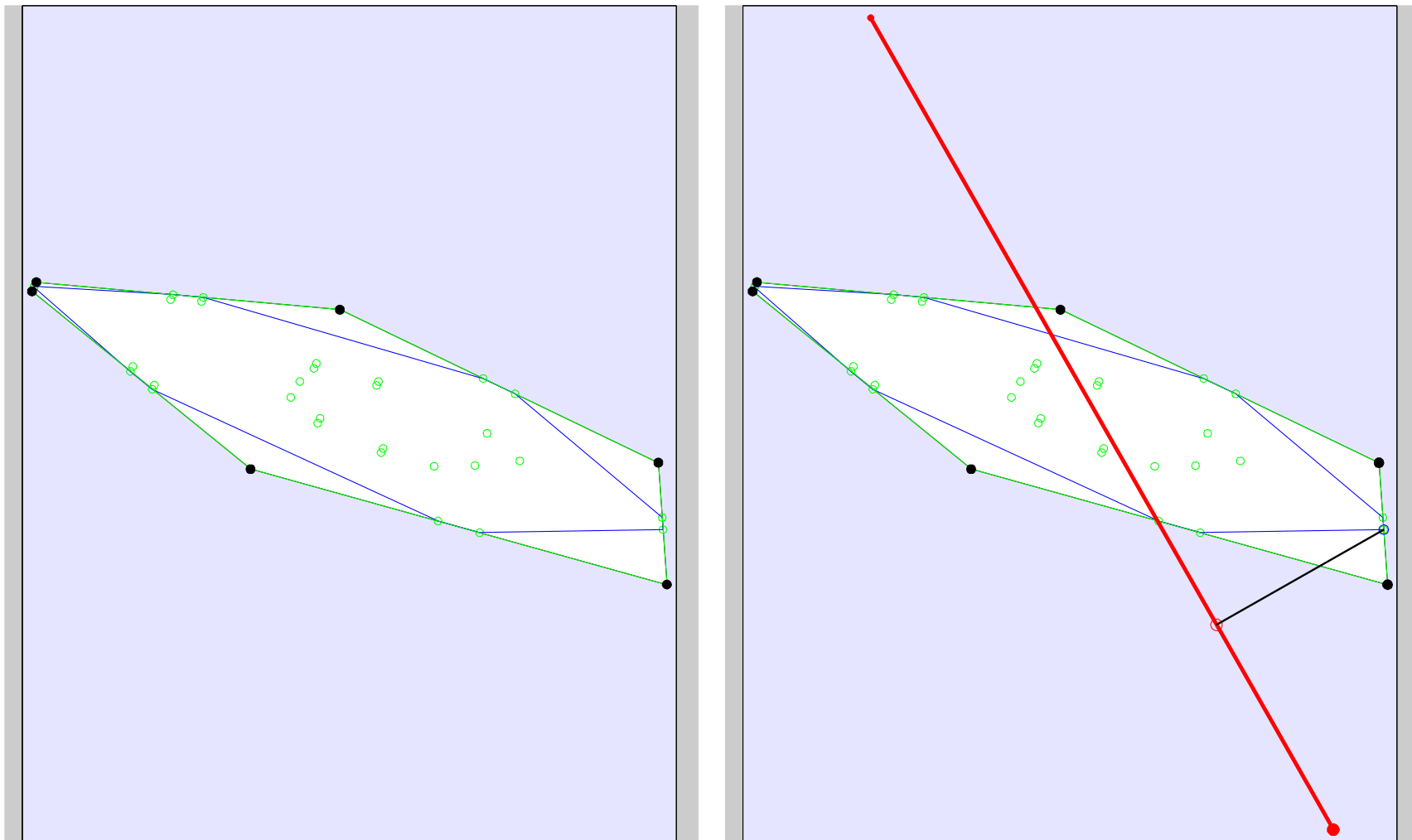
Minimum Projection (1/2)

- Algorithm

- 1) Obtain the projections of the original points onto the given direction.
- 2) Sort these projections in ascending order and choose the $\lceil 1/\mu \rceil$ first of them.
- 3) Take the weighted average of the selected ordered projections, with weight μ for the $\lfloor 1/\mu \rfloor$ smallest projections and $1 - \lfloor 1/\mu \rfloor \mu$ (if it is nonzero) for the last ($\lceil 1/\mu \rceil$ th) projection. The result is the projection of the extreme RCH point, required in the algorithmic computations. Note also that knowing the projection, the respective extreme point can easily be derived, if it is required.

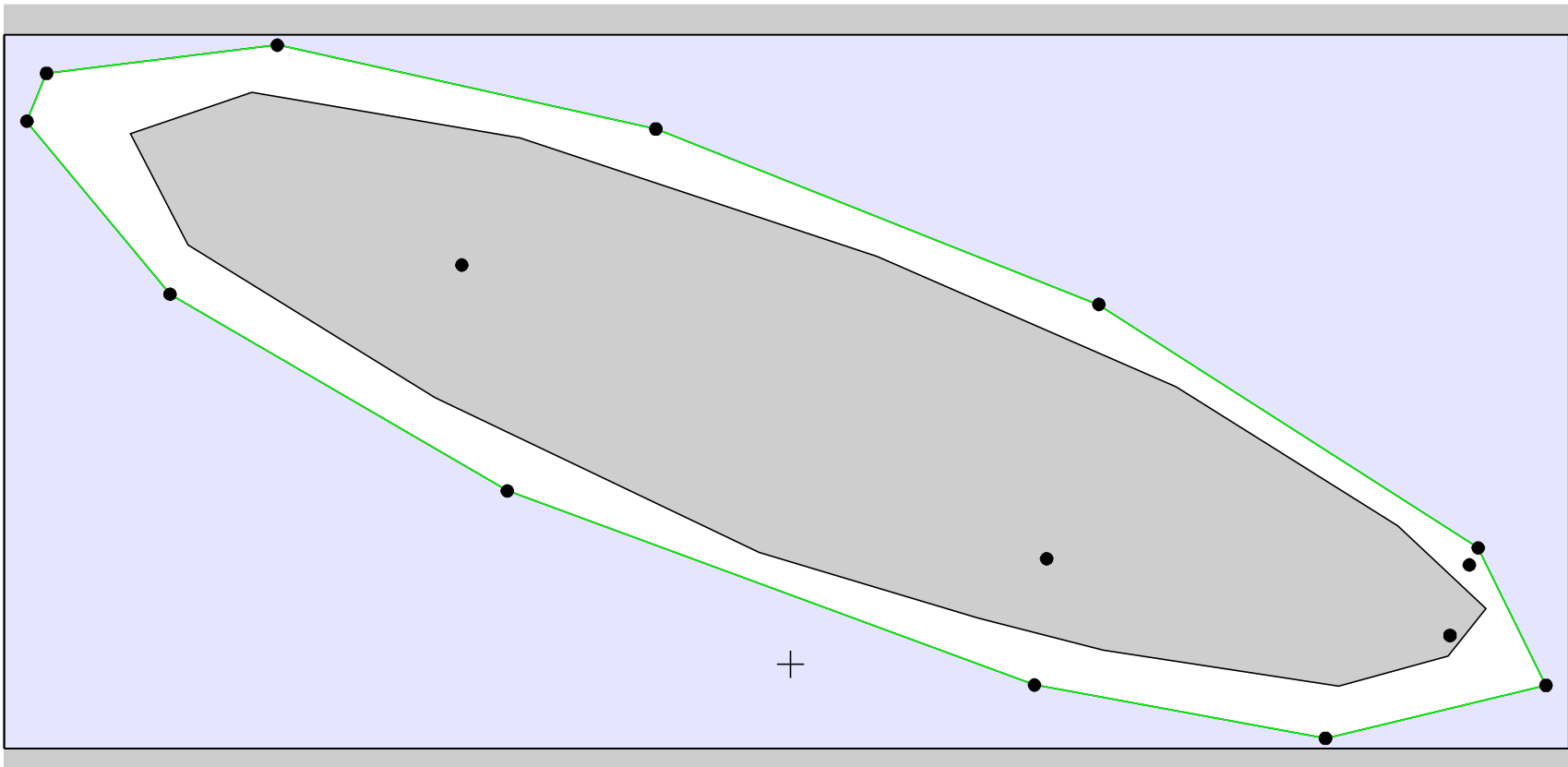
Minimum Projection (2/2)

- Candidates to be extreme points



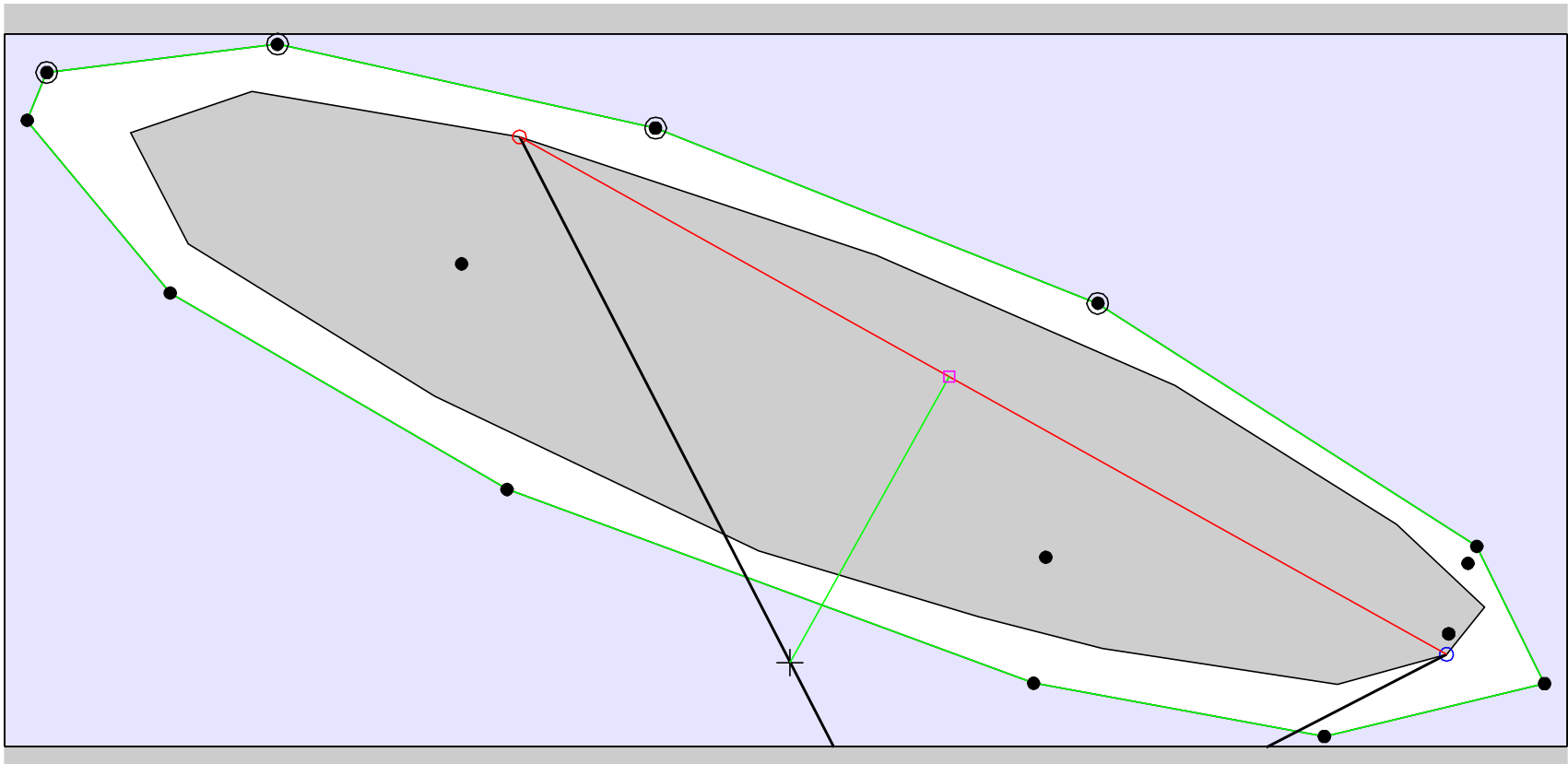
Gilbert's Algorithm Example (1/23)

- $\text{RCH}(X, 0.25)$



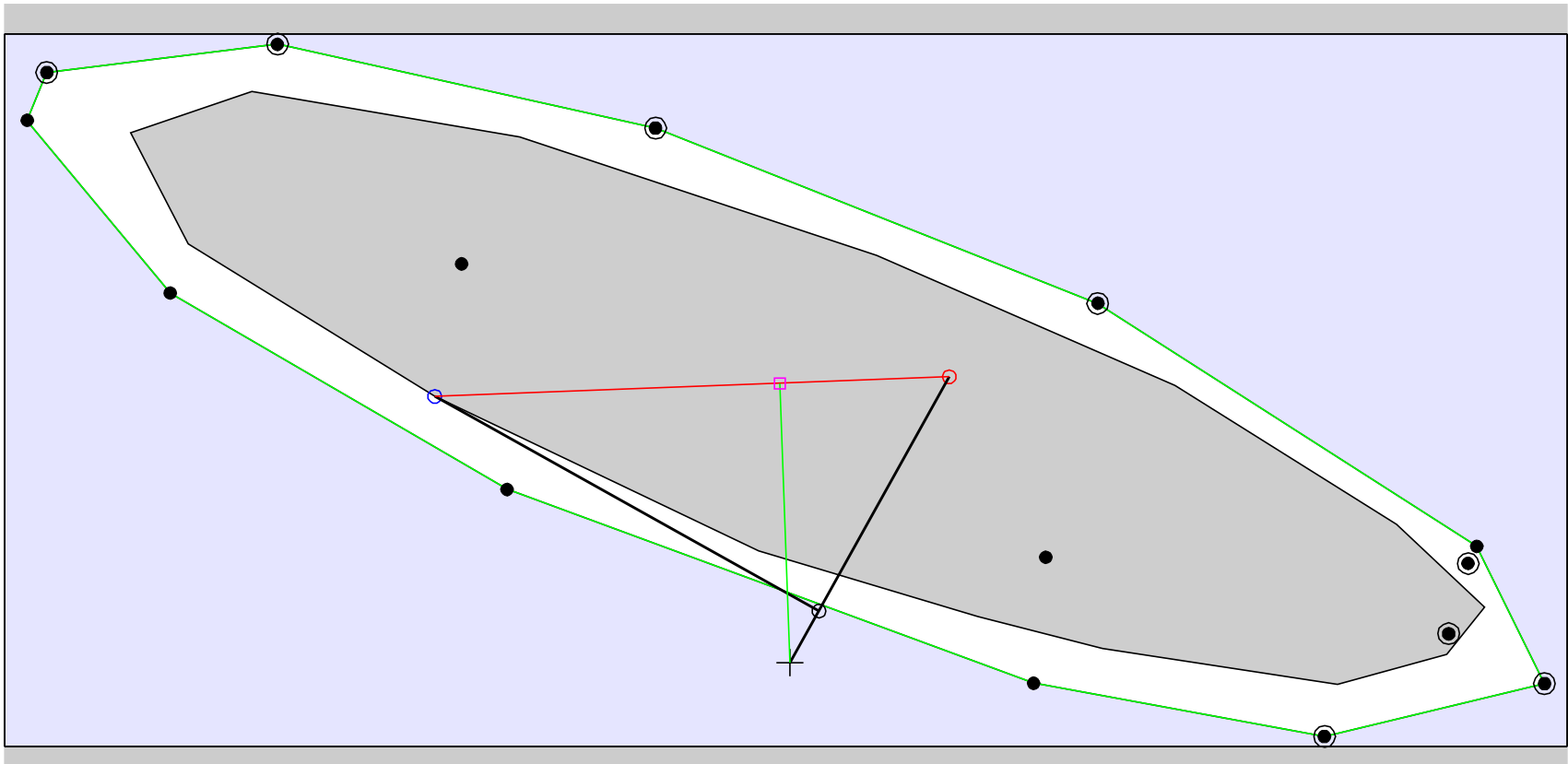
Gilbert's Algorithm Example (2/23)

- Step 1



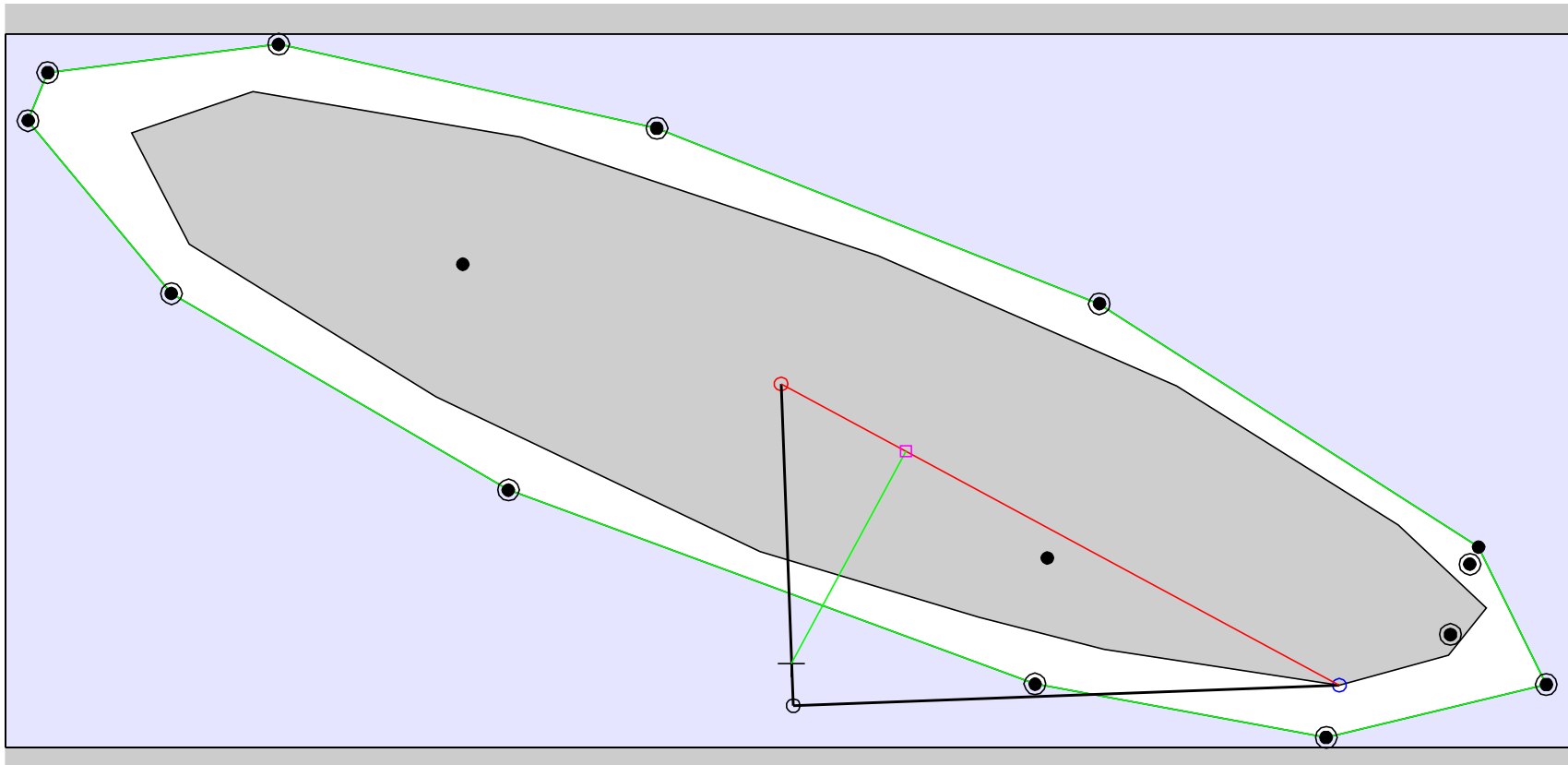
Gilbert's Algorithm Example (3/23)

- Step 2



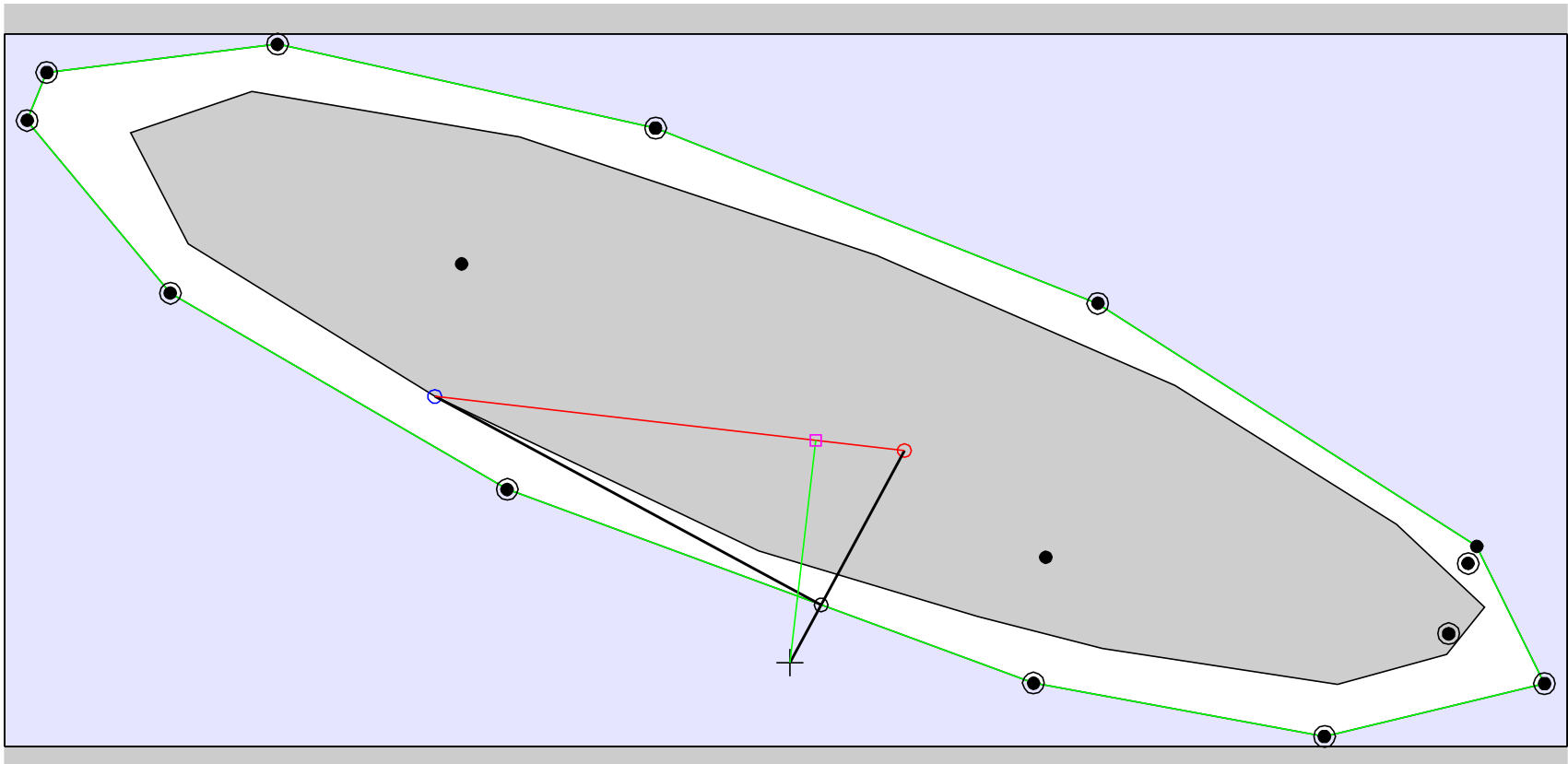
Gilbert's Algorithm Example (4/23)

- Step 3



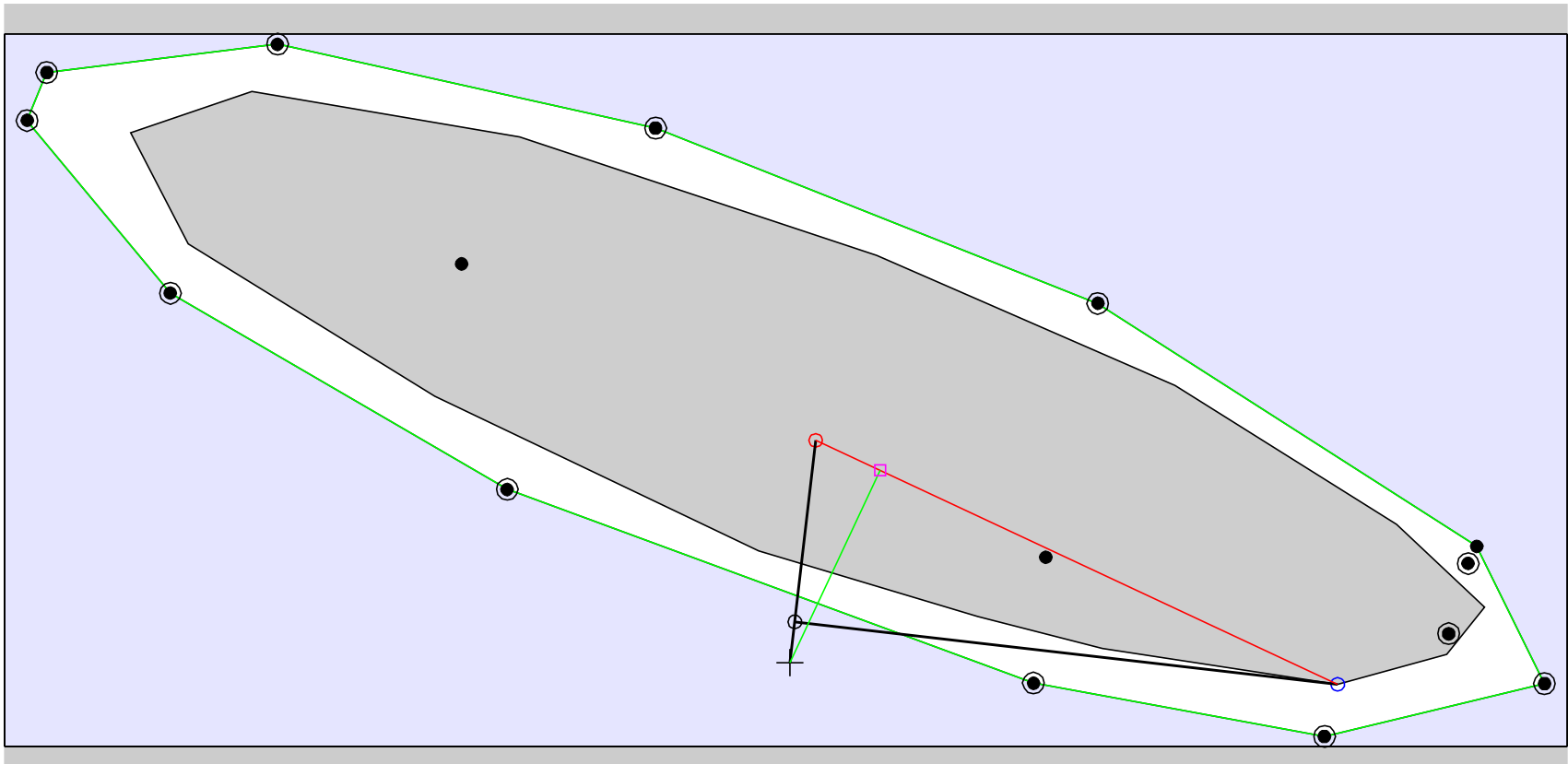
Gilbert's Algorithm Example (5/23)

- Step 4



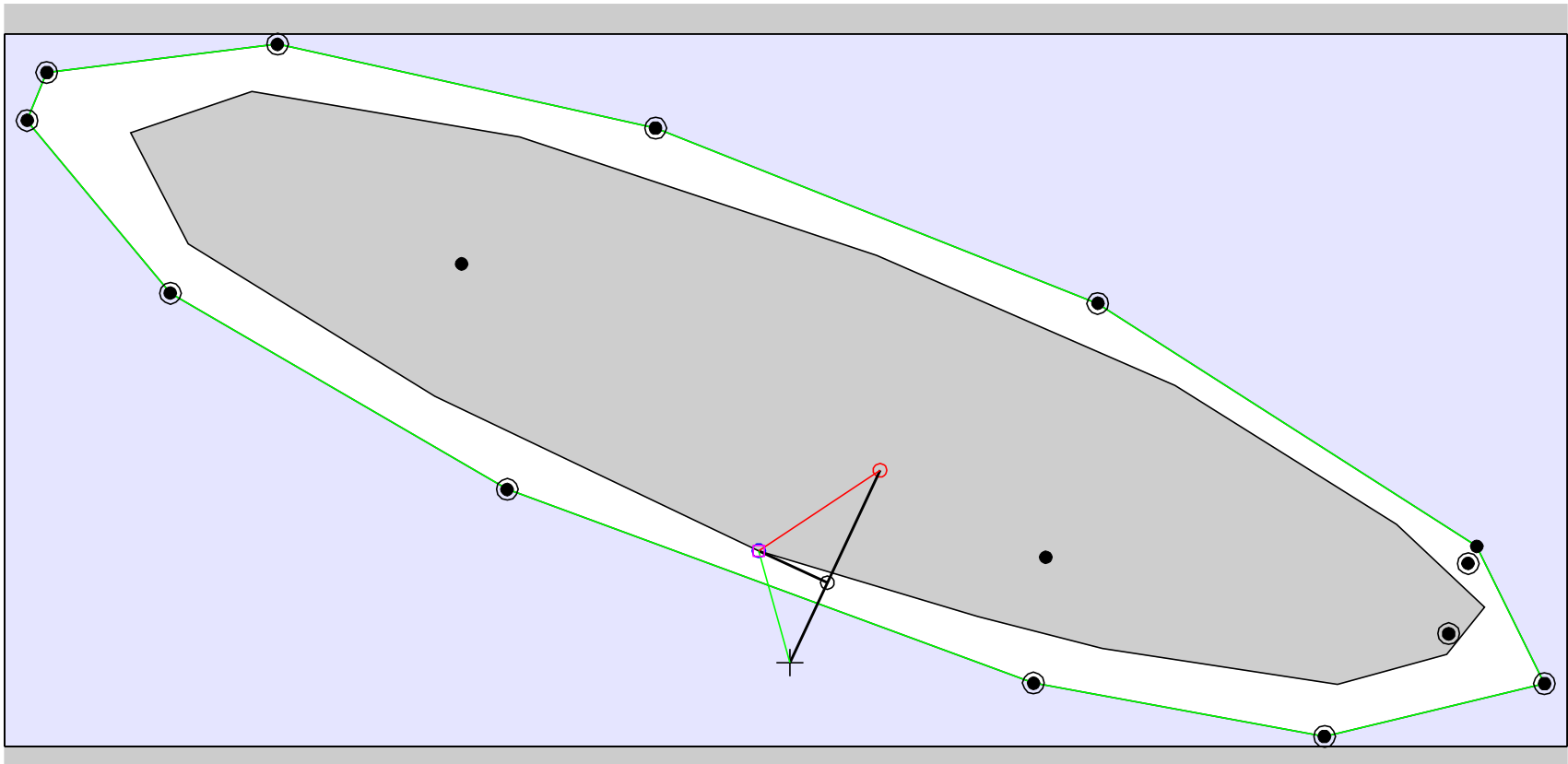
Gilbert's Algorithm Example (6/23)

- Step 5



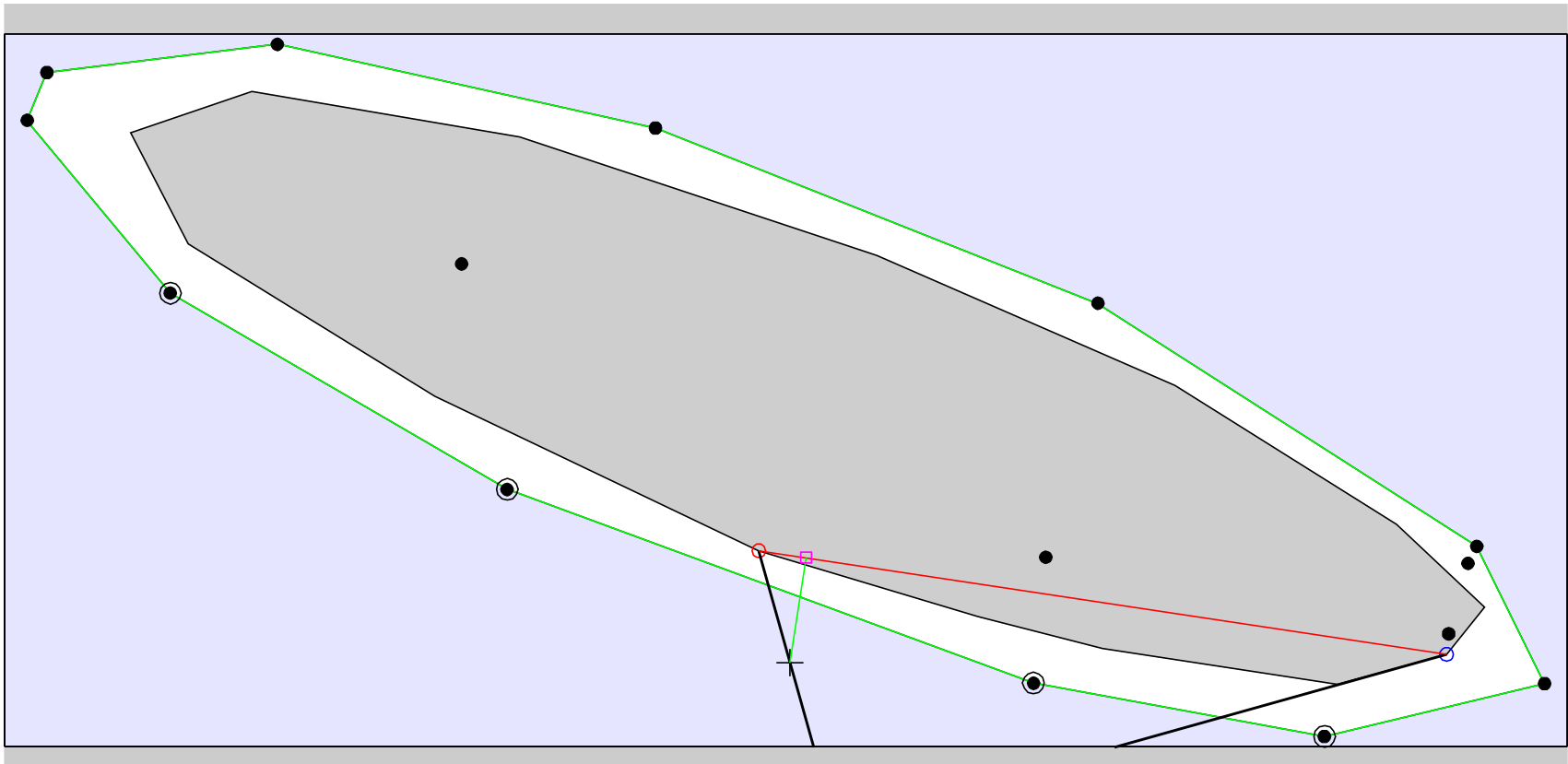
Gilbert's Algorithm Example (7/23)

- Step 6



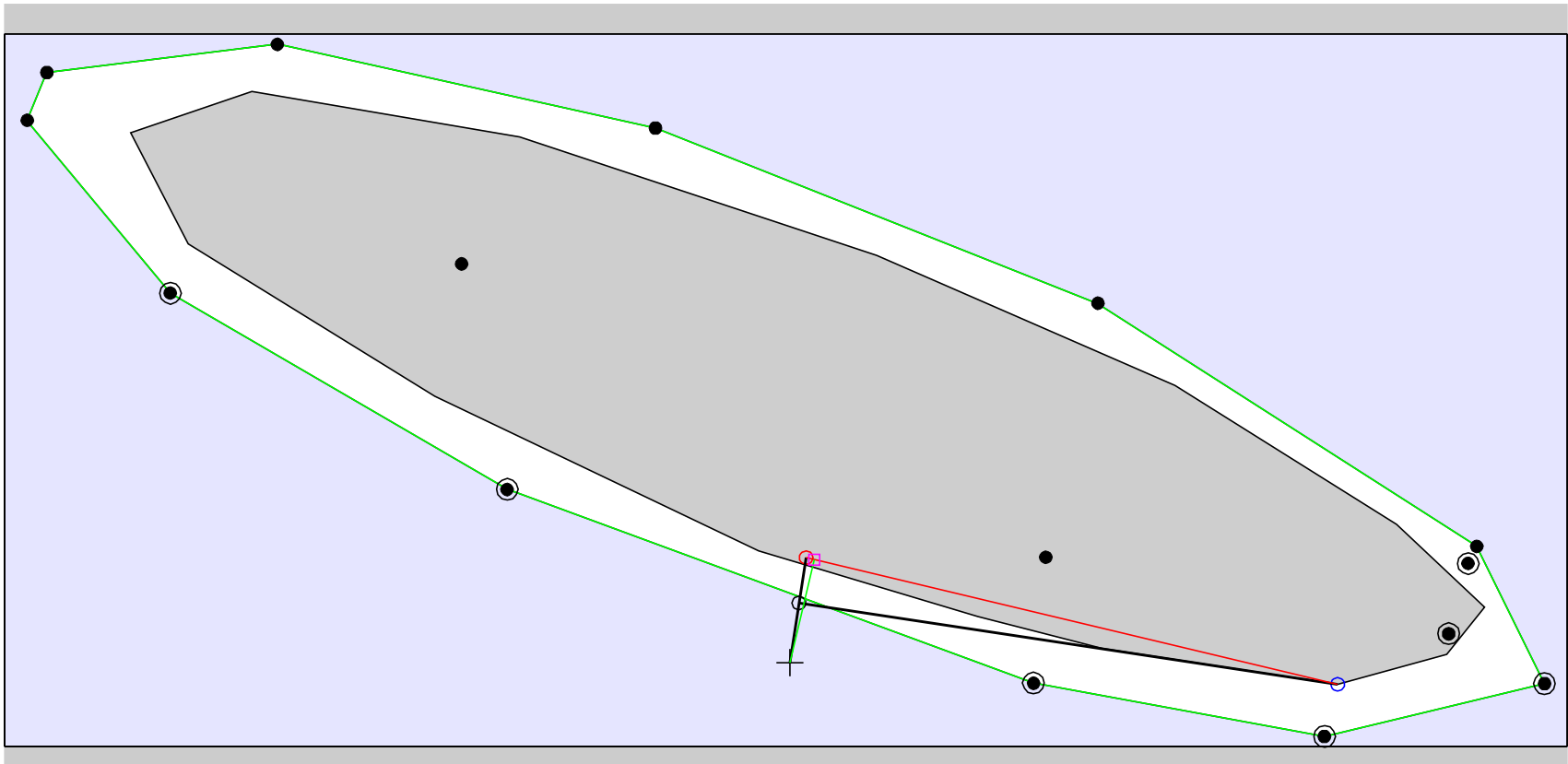
Gilbert's Algorithm Example (8/23)

- Step 7



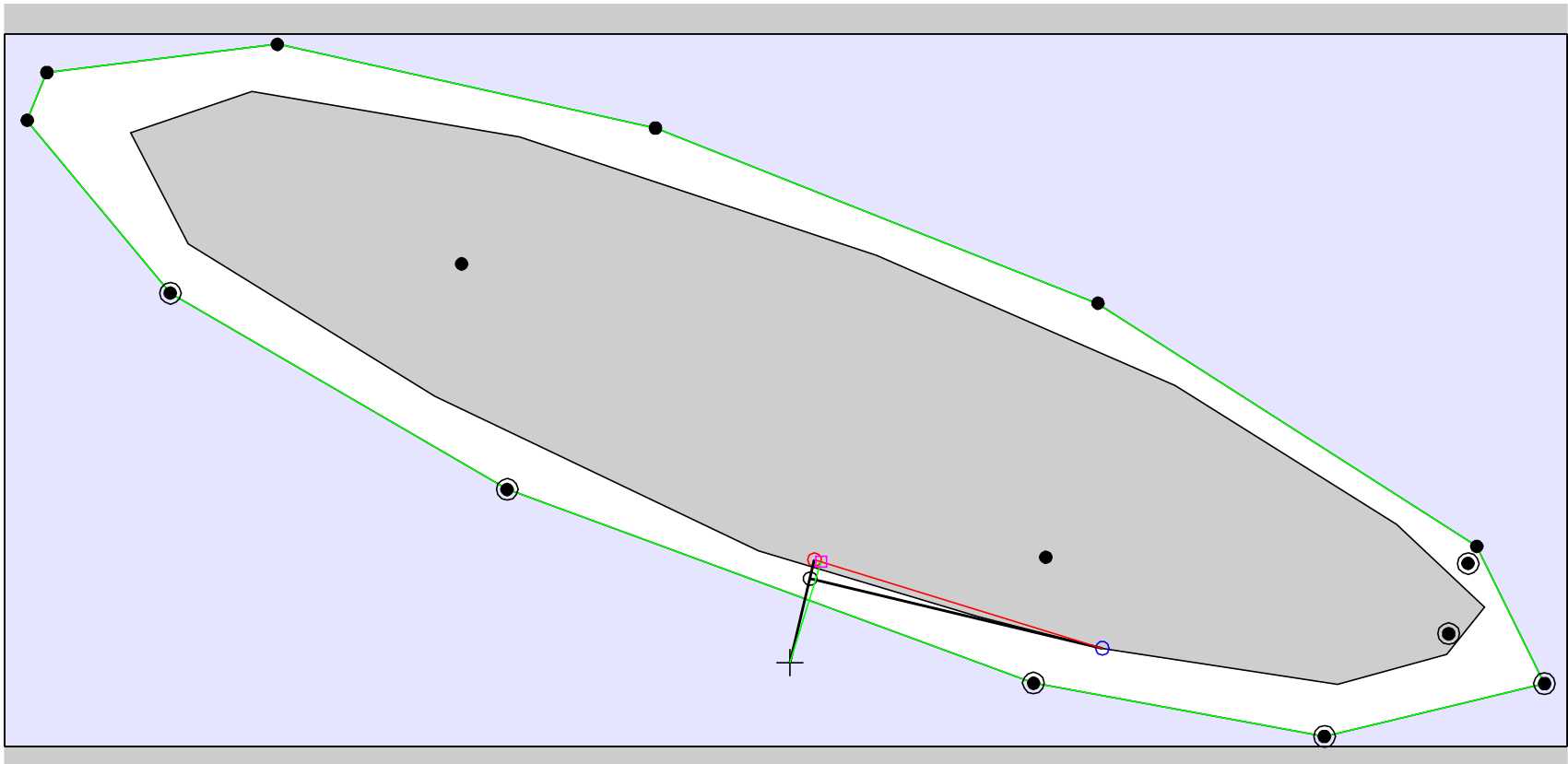
Gilbert's Algorithm Example (9/23)

- Step 8



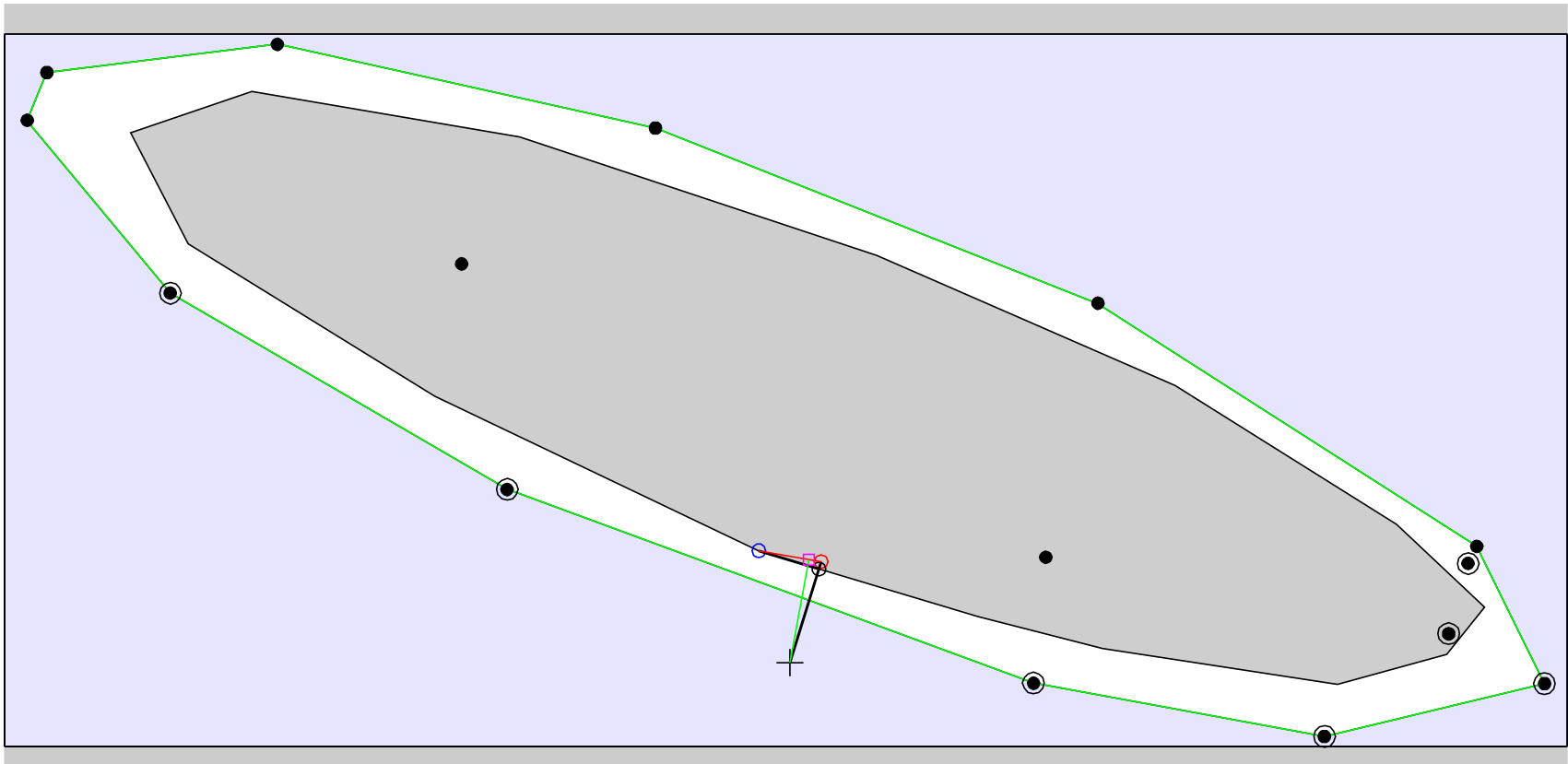
Gilbert's Algorithm Example (10/23)

- Step 9



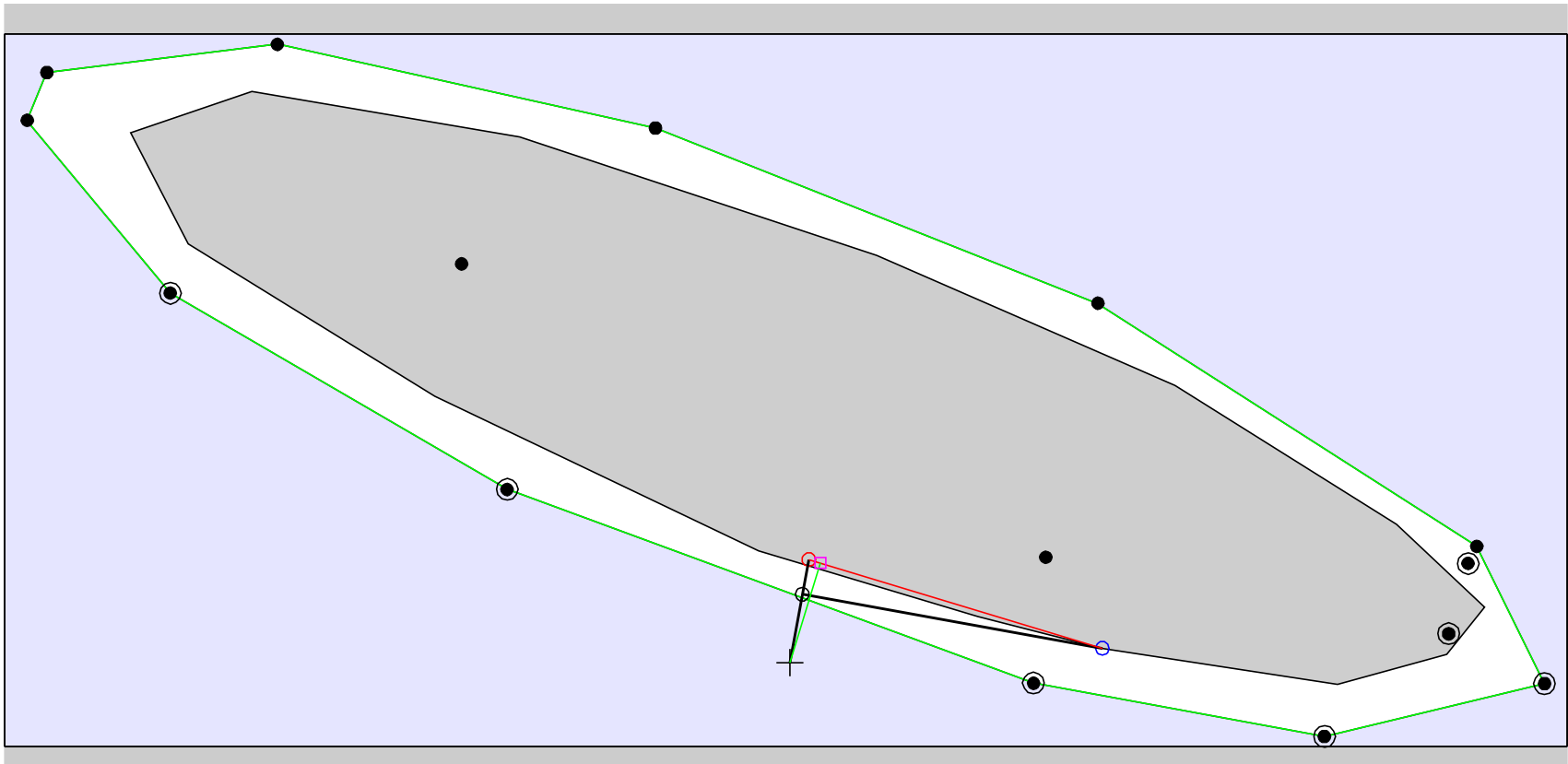
Gilbert's Algorithm Example (11/23)

- Step 10



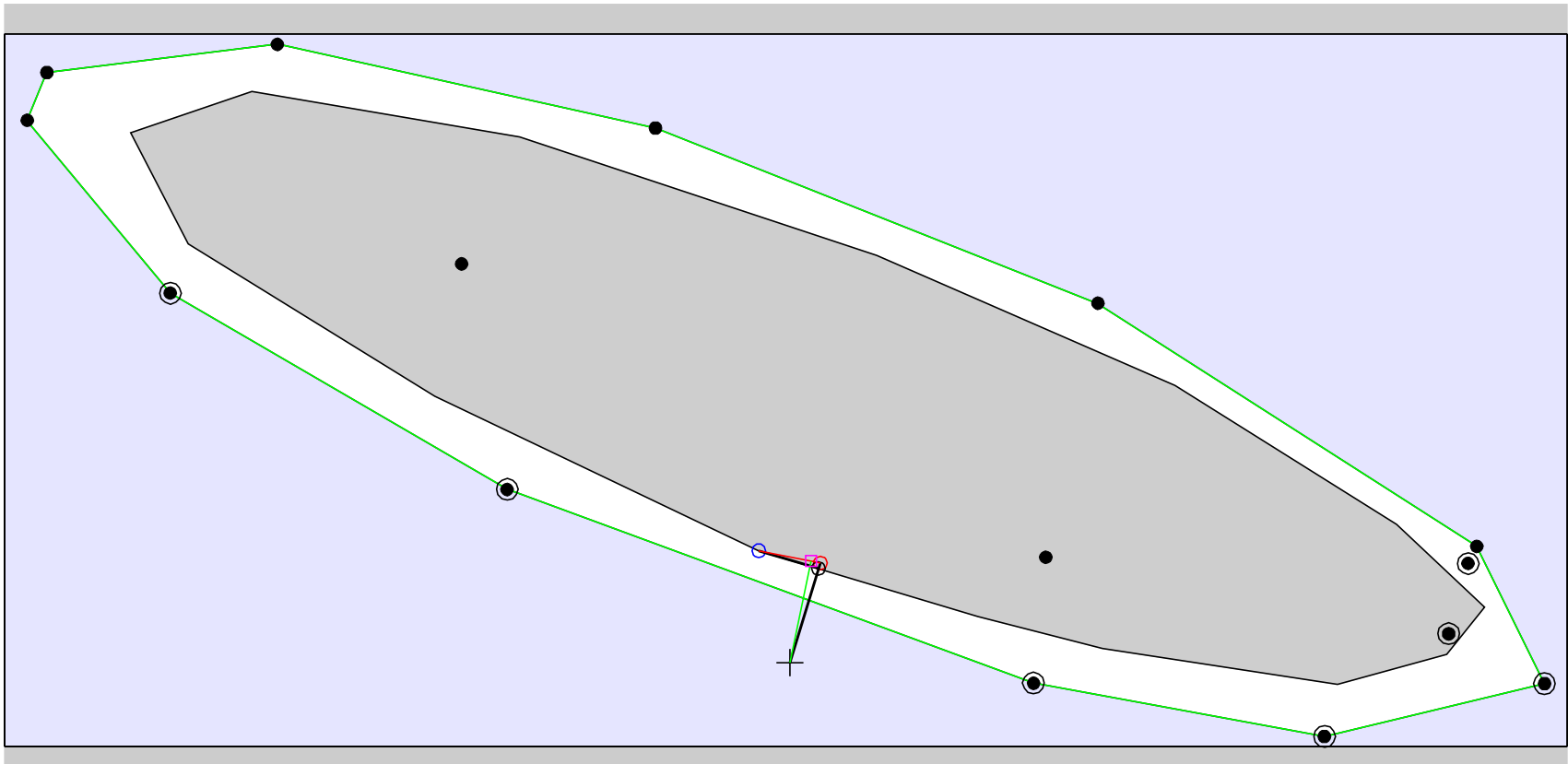
Gilbert's Algorithm Example (12/23)

- Step 11



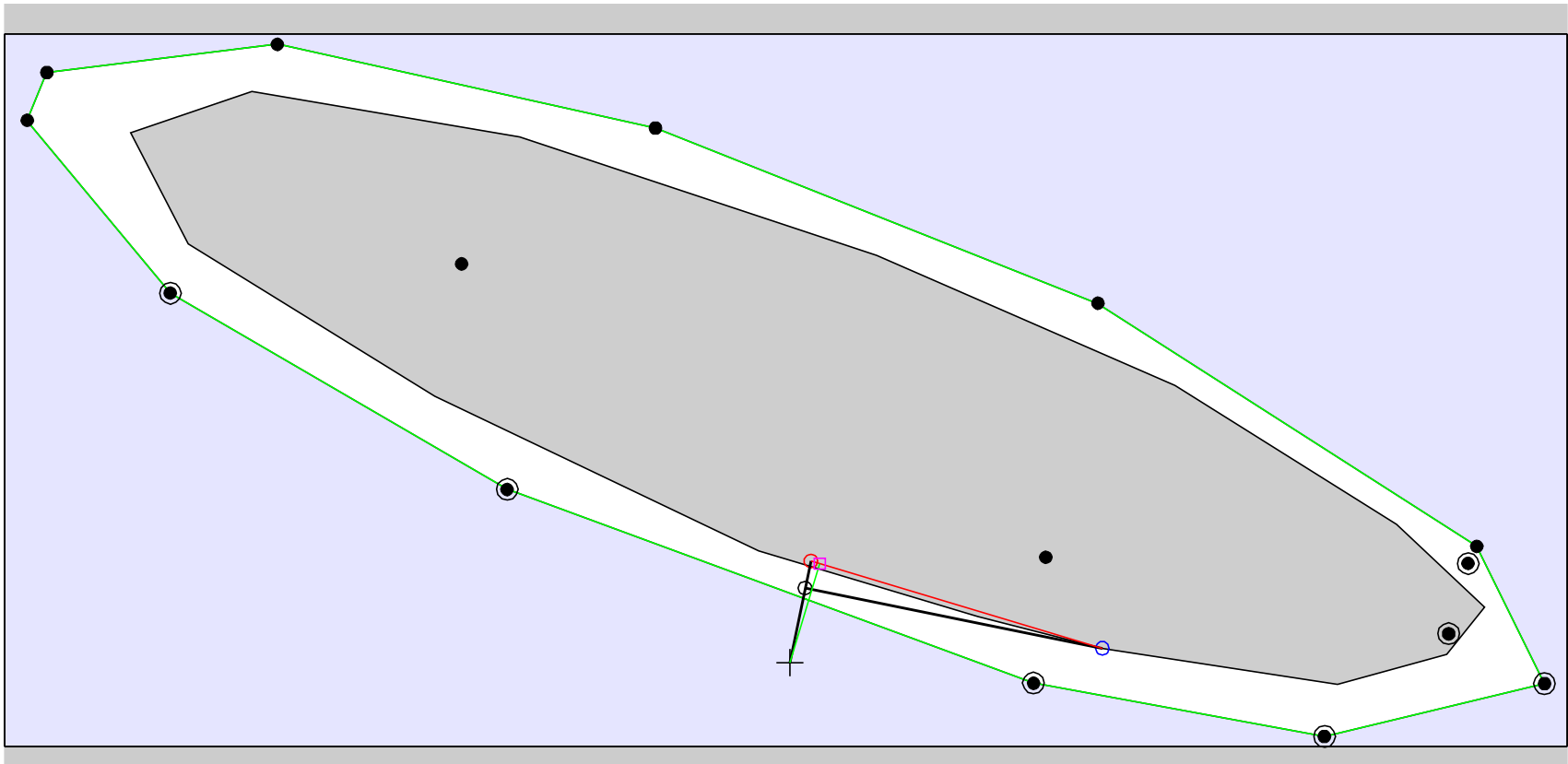
Gilbert's Algorithm Example (13/23)

- Step 12



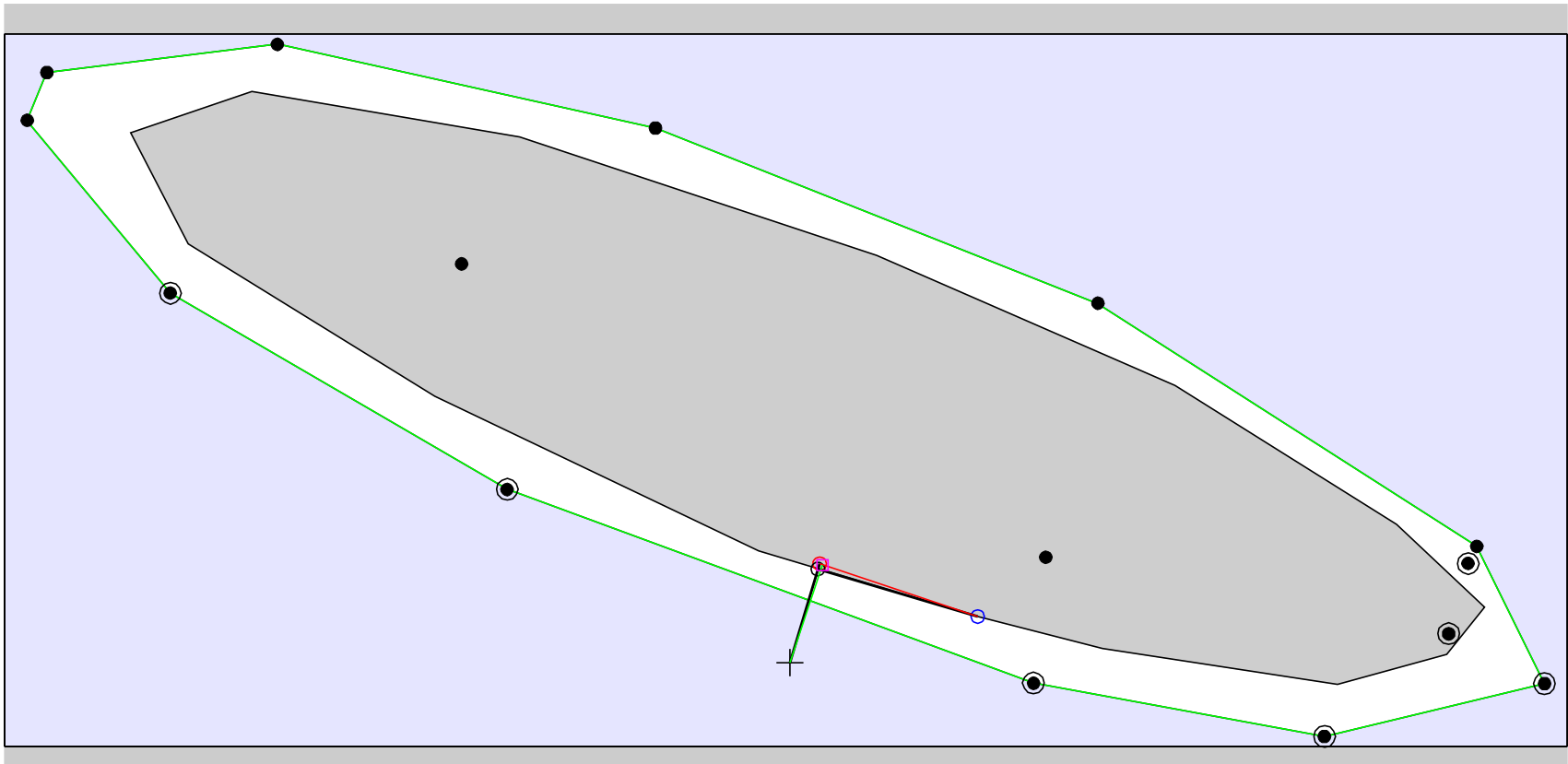
Gilbert's Algorithm Example (14/23)

- Step 13



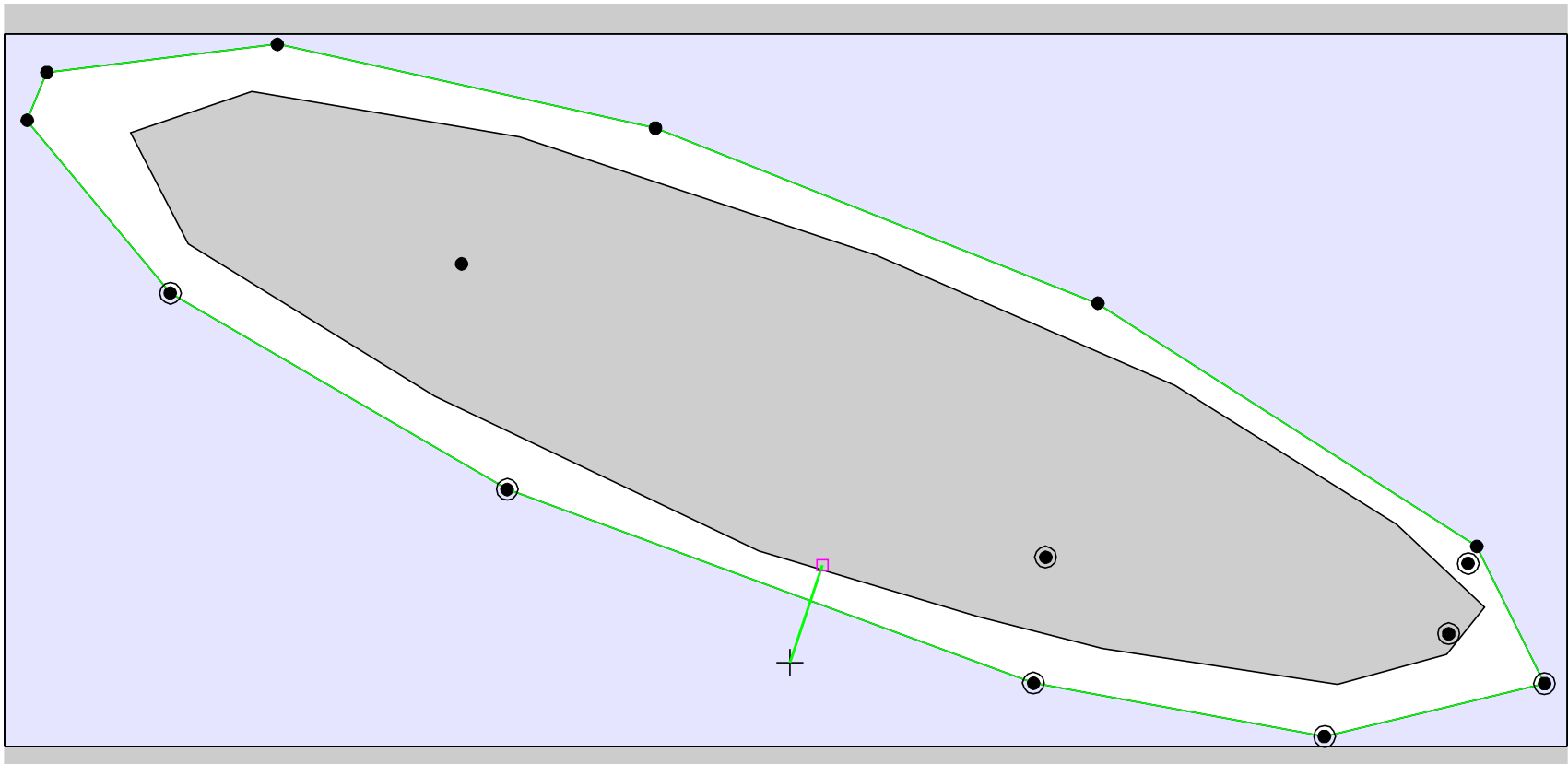
Gilbert's Algorithm Example (15/23)

- Step 14

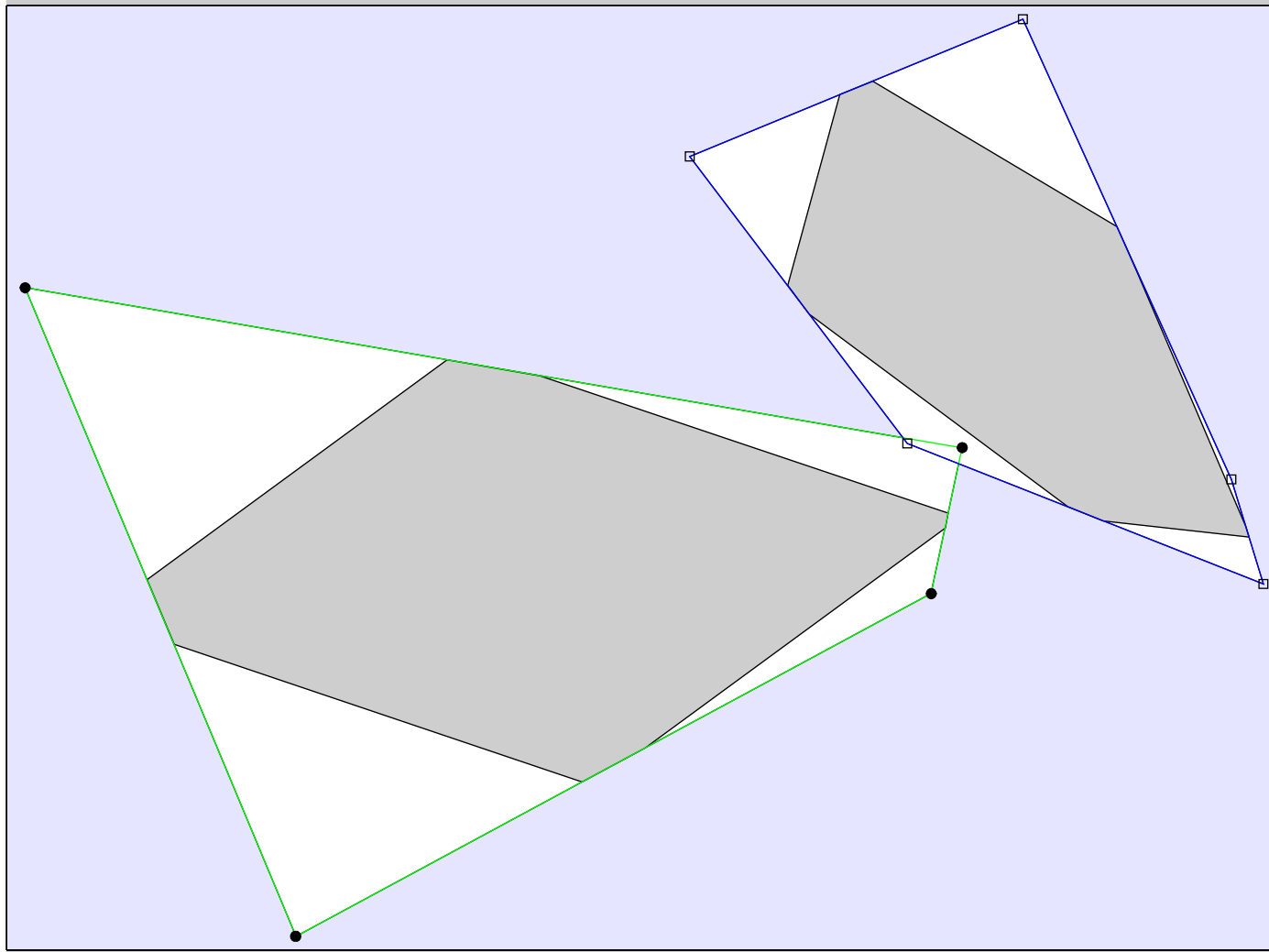


Gilbert's Algorithm Example (16/23)

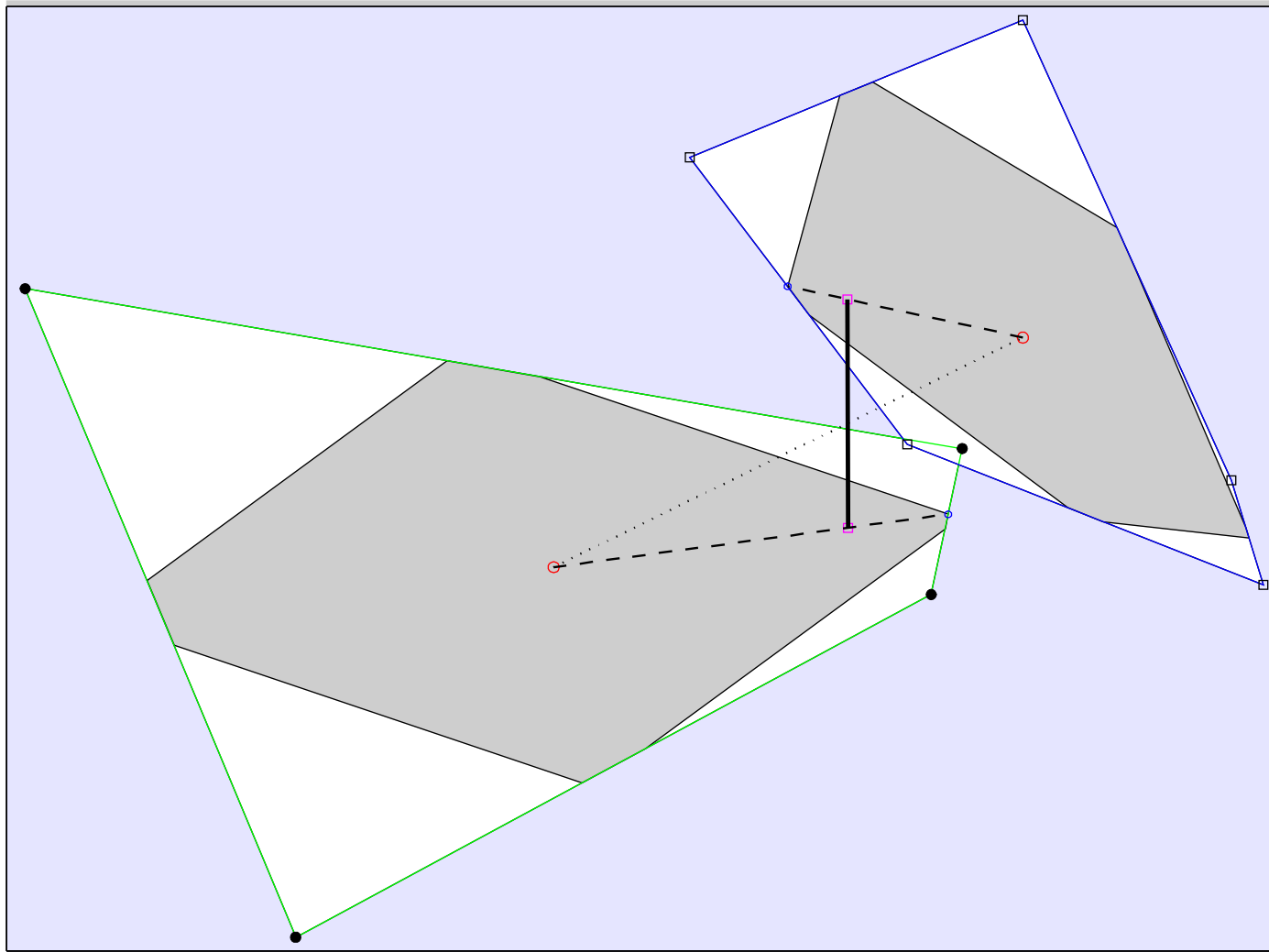
- Step 15



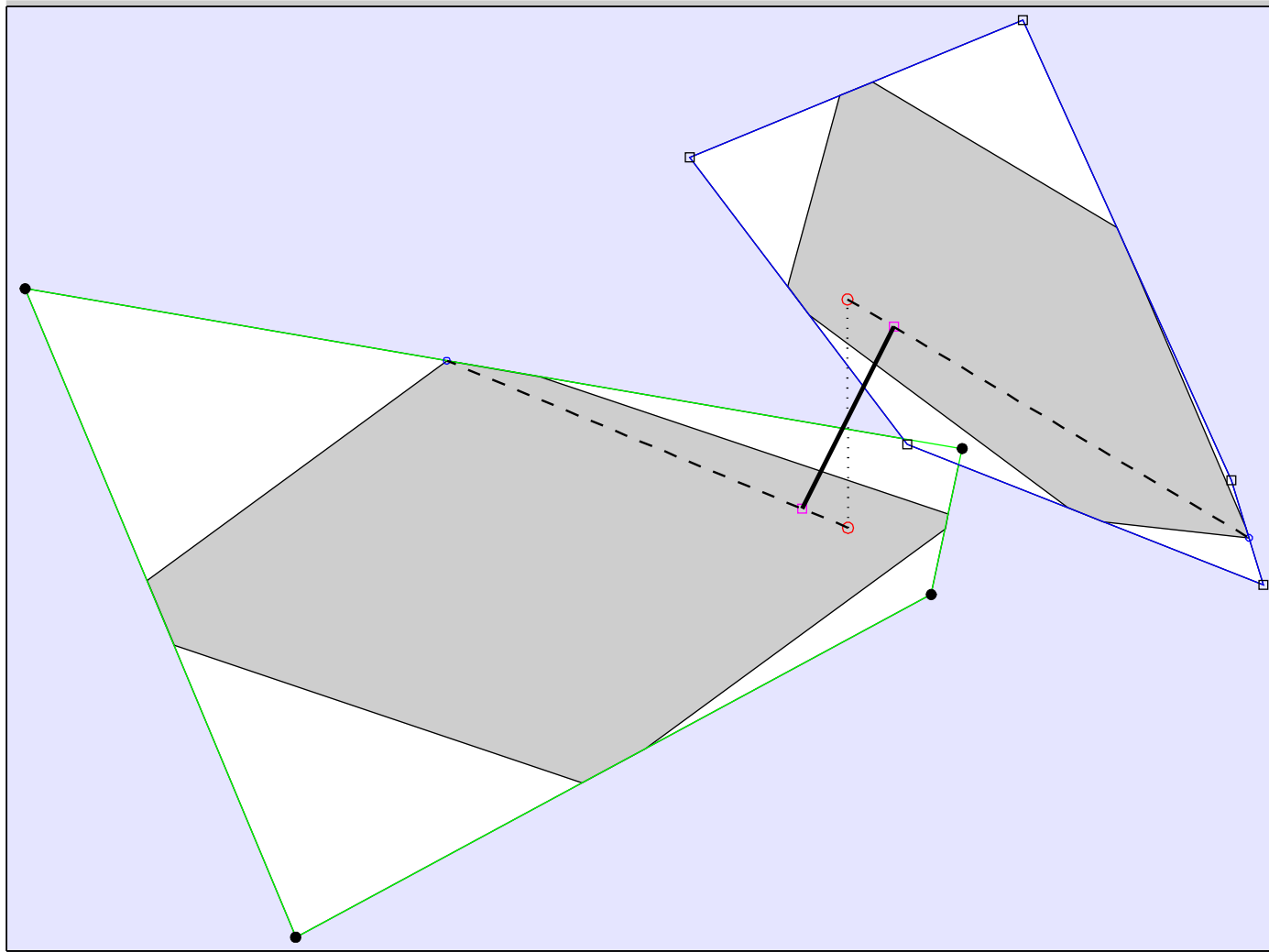
Gilbert's Algorithm Example (17/23)



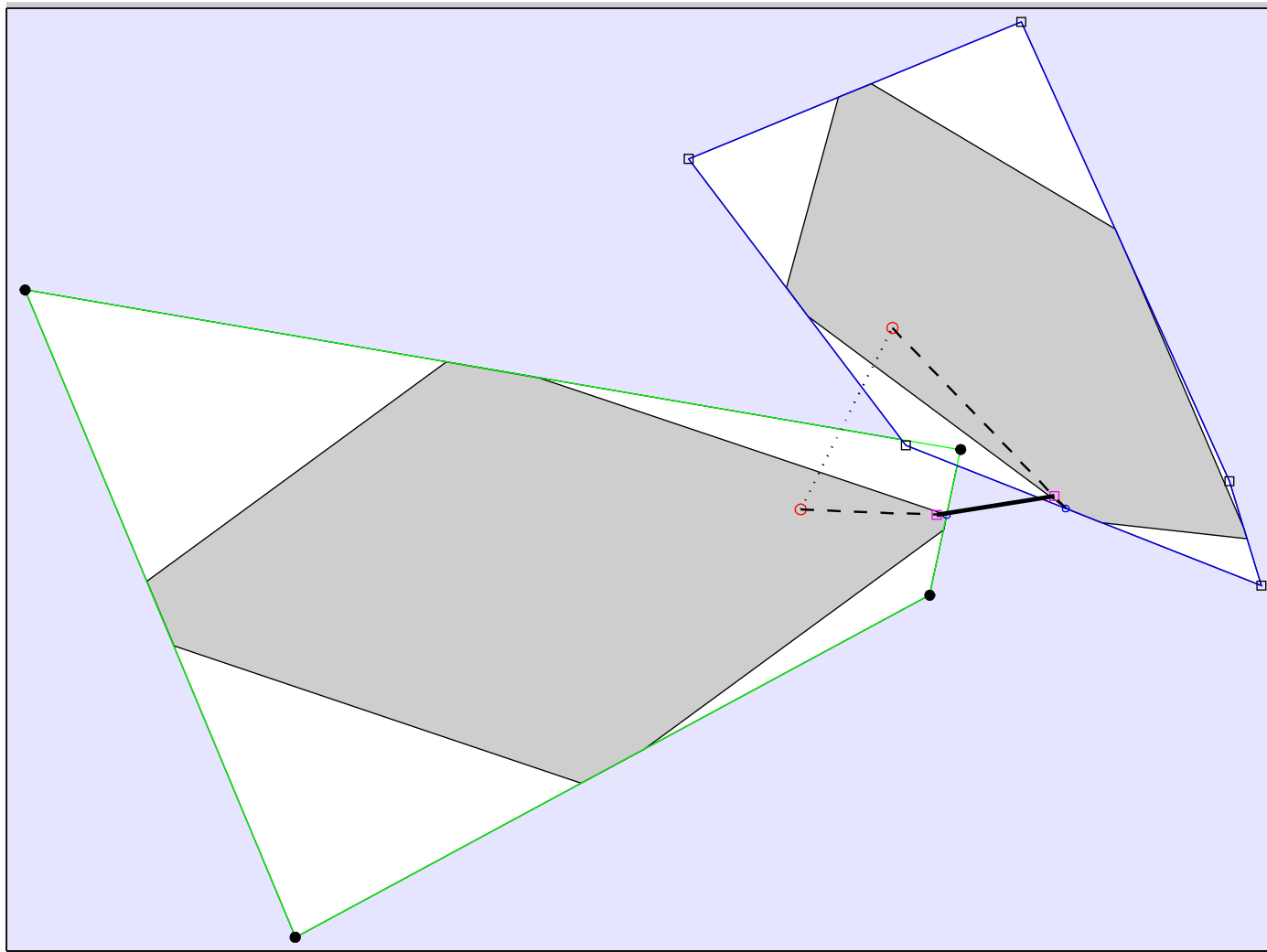
Gilbert's Algorithm Example (18/23)



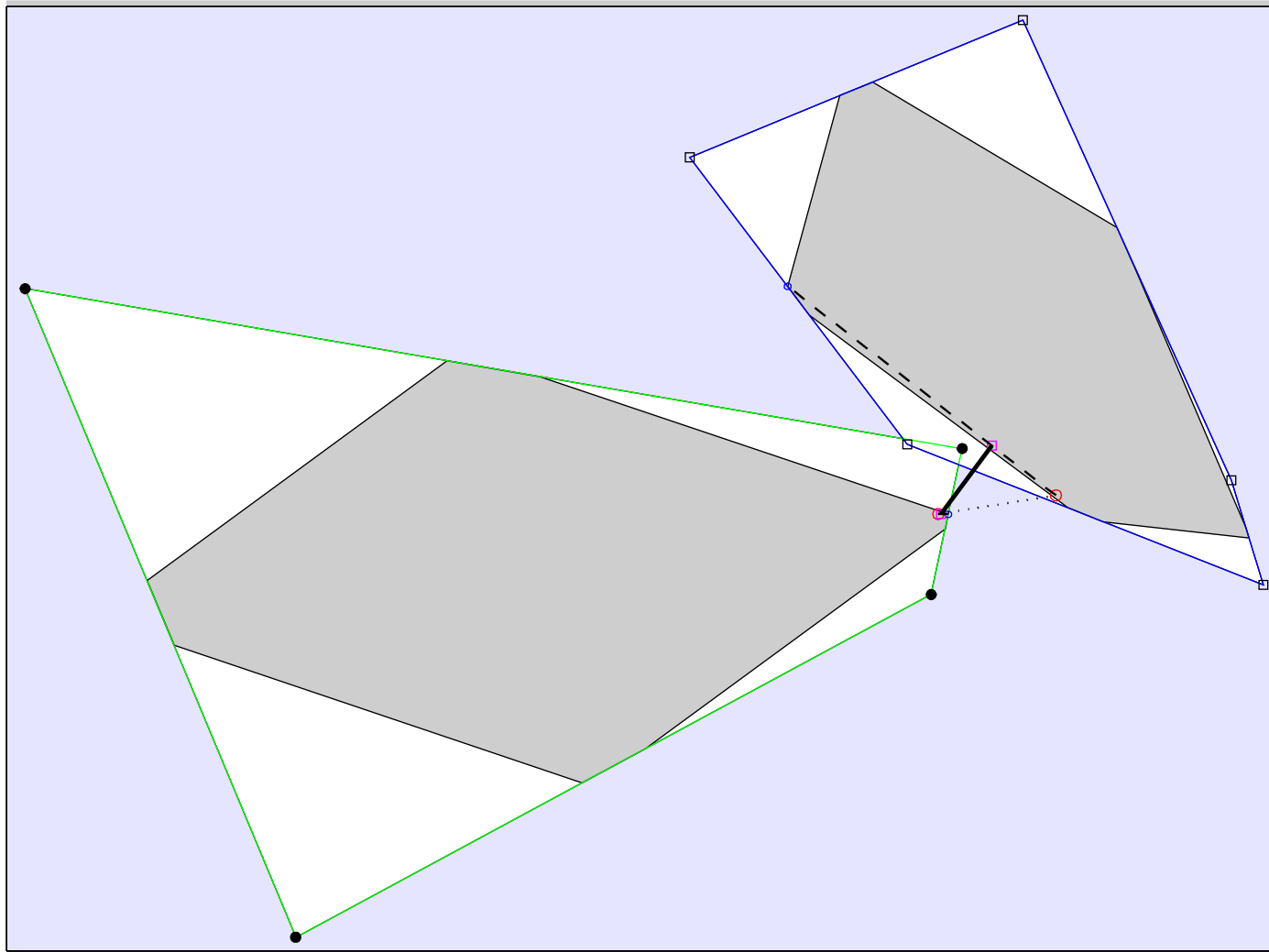
Gilbert's Algorithm Example (19/23)



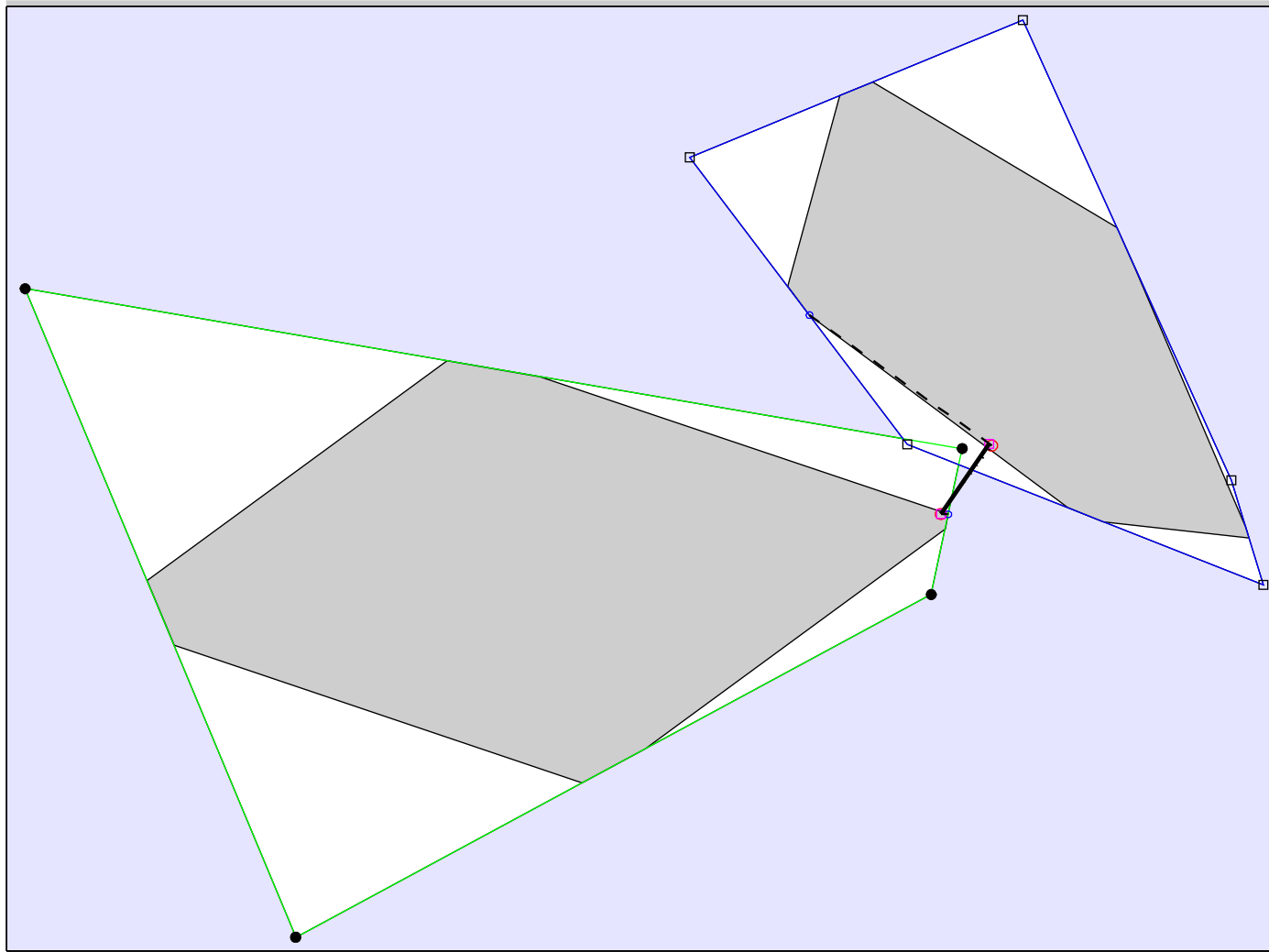
Gilbert's Algorithm Example (20/23)



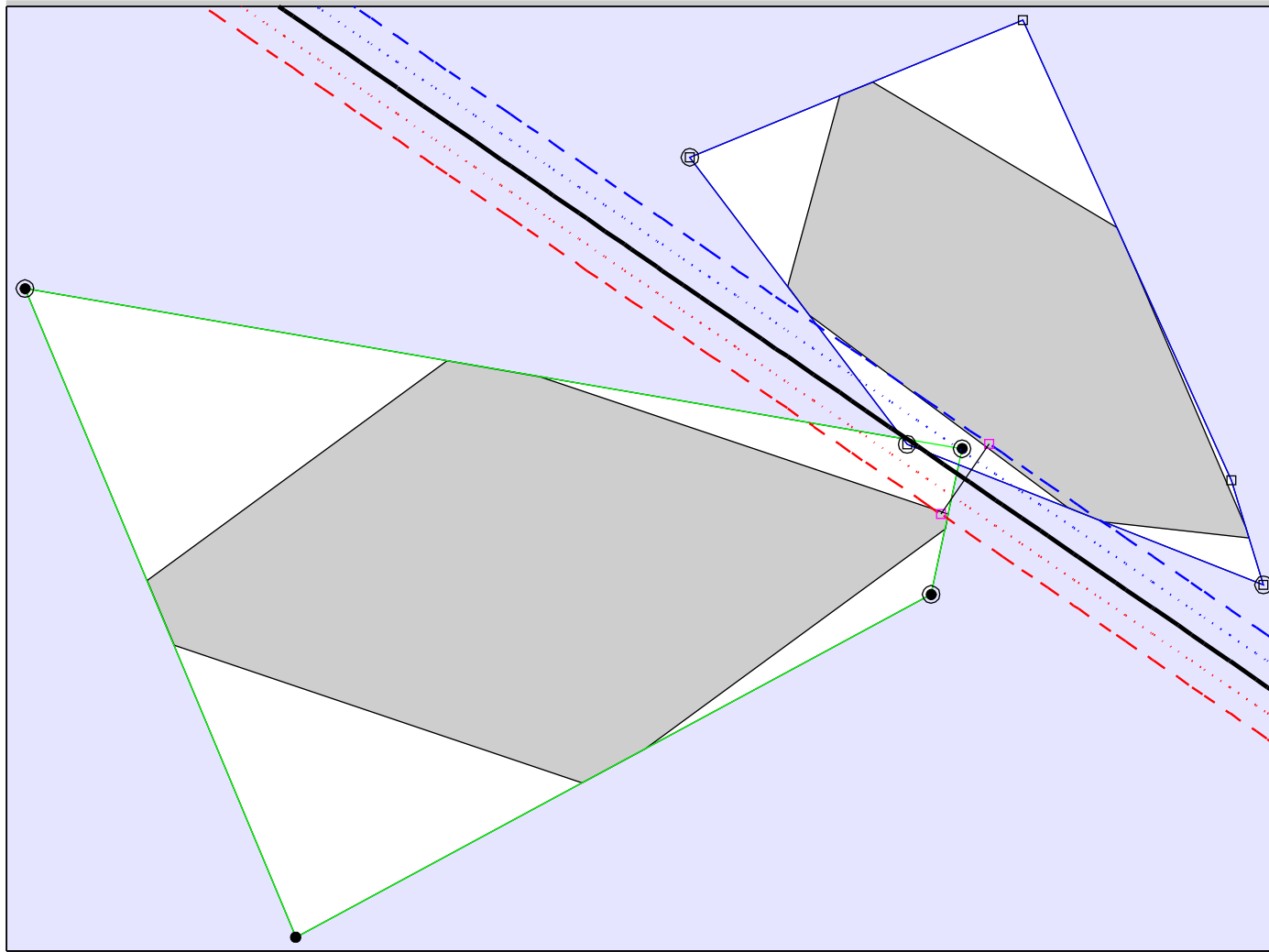
Gilbert's Algorithm Example (21/23)



Gilbert's Algorithm Example (22/23)



Gilbert's Algorithm Example (23/23)



Experimental (1/3)

Dataset	Data Dim.	Train Patterns	Test Patterns	Success Reported
Diabetis	8	468	300	76.5±1.7
Banana	2	400	4900	88.5±0.7
German	20	700	300	76.4±2.1
Wave-form	21	400	4600	90.1±0.4
Flare-Solar	9	666	400	67.6±1.8
Heart	13	170	100	84.0±3.3

Experimental (2/3)

Dataset	σ	C , tolerance	μ , accuracy
Diabetis	20	100, 10^{-3}	0.0075, 10^{-3}
Banana	1	316.2, 10^{-3}	0.014, 5×10^{-4}
German	55	3162, 10^{-3}	0.0052, 10^{-3}
Wave-form	20	1000, 10^{-3}	0.02, 5×10^{-4}
Flare-Solar	30	1000, 10^{-3}	0.0039, 10^{-3}
Heart	120	1000, 10^{-3}	0.017, 10^{-3}

Experimental (3/3)

Dataset	Method	Success (%)	Kernel Eval. ($\times 10^6$)	Dataset	Method	Success (%)	Kernel Eval. ($\times 10^6$)
Diabetis	SMO	76.7 $\pm$ 1.8	20.10	Waveform	SMO	88.8 $\pm$ 0.6	94.30
	SMO-K1	76.7$\pm$1.8	1.45		SMO-K1	89.2$\pm$0.5	2.20
	SMO-K2	76.7 $\pm$ 1.8	4.10		SMO-K2	88.9 $\pm$ 1.4	6.20
	RCH-SK	76.2 $\pm$ 1.8	0.14		RCH-SK	87.8 $\pm$ 2.3	3.81
	RCH-G	76.3 $\pm$ 1.8	0.70		RCH-G	88.5 $\pm$ 0.7	1.46
Banana	SMO	89.1$\pm$0.6	564.33	Flare-Solar	SMO	67.5 $\pm$ 1.8	22.80
	SMO-K1	89.1 $\pm$ 0.6	6.60		SMO-K1	67.6 $\pm$ 1.8	11.90
	SMO-K2	89.0 $\pm$ 0.7	15.40		SMO-K2	67.6 $\pm$ 1.8	18.60
	RCH-SK	89.0 $\pm$ 0.6	14.27		RCH-SK	67.6 $\pm$ 1.7	2.08
	RCH-G	88.8 $\pm$ 0.9	1.75		RCH-G	67.7$\pm$1.8	3.37
German	SMO	76.0 $\pm$ 2.2	13.70	Heart	SMO	83.9 $\pm$ 3.3	2.05
	SMO-K1	76.1 $\pm$ 2.2	9.00		SMO-K1	83.2 $\pm$ 4.2	0.26
	SMO-K2	76.1 $\pm$ 2.2	46.20		SMO-K2	83.4 $\pm$ 4.2	0.26
	RCH-SK	75.5 $\pm$ 0.2	3.62		RCH-SK	84.2 $\pm$ 2.7	7.50
	RCH-G	76.3$\pm$0.4	2.71		RCH-G	84.8$\pm$3.3	0.05

Conclusion

- Gilbert's algorithm involves only inner product operations and it can be readily kernelised. It is thus appropriate for the general case of non-linear and non-separable class tasks.
- The concept of RCH has led to a new framework for developing geometric algorithms for the SVM problem.
- The algorithms are of a strong intuitive nature and computationally efficient.